Pricing, Greeks, and Jump-Risk Management for BTC/ETH Vanilla Options

IEDA4331 Financial Engineering Group Project

Choi Man Hou
mhchoiaf@connect.ust.hk
SID: 20894196

Lee Cheuk Yiu
cyleebt@connect.ust.hk
SID: 21051040

Market data snapshot: 2026-05-15 09:06 UTC

Abstract

This report prices BTC and ETH vanilla options using the core IEDA4331 toolkit: risk-neutral valuation, Black-Scholes pricing, CRR binomial trees, implied volatility, Greeks, and dynamic hedging. We treat the Merton jump-diffusion model as a classical jump benchmark rather than the final innovation. The modern exploration layer then evaluates four crypto-relevant candidates at the same empirical level: stochastic volatility with correlated jumps, market-regime implied stochastic volatility, time-varying jump intensity, and GARCH option pricing. The calibrated Merton benchmark reduces mean absolute pricing error by 68.9% for BTC and 48.3% for ETH relative to a historical-volatility Black-Scholes baseline. The refreshed equal-level candidate comparison is deliberately nuanced: Merton gives the lowest BTC MAE, MR-ISVM gives the lowest ETH MAE, and SVCJ produces the largest standardized crash-protection premium. The main conclusion is that crypto option markets price non-Gaussian jump and volatility-regime risk explicitly; Black-Scholes remains a necessary no-arbitrage benchmark, but it is not sufficient as a standalone crypto risk engine.

Contents

1 Introduction and Course Motivation

This report studies BTC and ETH vanilla options under the central language of IEDA4331: no-arbitrage pricing, risk-neutral valuation, binomial replication, the Black-Scholes-Merton PDE, implied volatility, Greeks, and hedging error. The objective is to move from classroom pricing identities to a market-calibrated crypto option risk engine, then ask where the classical diffusion model fails. We use the Black-Scholes model [2, 8], CRR binomial tree [4], and Merton jump-diffusion model [9] as the classical ladder of benchmarks.

The Risk-Neutral Pricing chapter brings the analysis from physical expected returns to martingale valuation. Instead of asking whether BTC or ETH is expected to appreciate, we price contingent claims under Q:

$$V_t = E_t^Q\left[e^{-r(T-t)}V_T\right].$$

The Binomial Tree chapter brings the analysis to finite-state replication: a derivative is priced by constructing a self-financing portfolio whose future payoff matches the claim. The Black-Scholes-Merton chapter then takes the limiting continuous-time view and derives the PDE from a self-financing delta hedge:

$$\frac{\partial V}{\partial t} + rS\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S^2\frac{\partial^2 V}{\partial S^2} - rV = 0.$$

The Risk Management chapter brings the analysis from valuation to control: Delta, Gamma, Vega, discrete hedging error, VaR logic, and stress testing. Our report follows that course arc, then adds a jump process precisely where the diffusion assumption breaks down for cryptocurrency markets.

2 Data and Market Snapshot

We use public Deribit data for BTC and ETH: `get_book_summary_by_currency` for option books, `get_instruments` for contract metadata, and `get_index_chart_data` for one-year index histories. Deribit option premiums are quoted in coin units, so we convert them into USD:

$$\text{Option price}_{USD} = \text{Option price}_{coin} \times S.$$

Table 1: Deribit option universe after cleaning

Currency	Raw rows	Usable rows	Median spot	Median IV	Min days	Max days
BTC	934	748	80803.9	41.66%	1.95	315.0
ETH	740	516	2263.3	56.56%	1.95	315.0

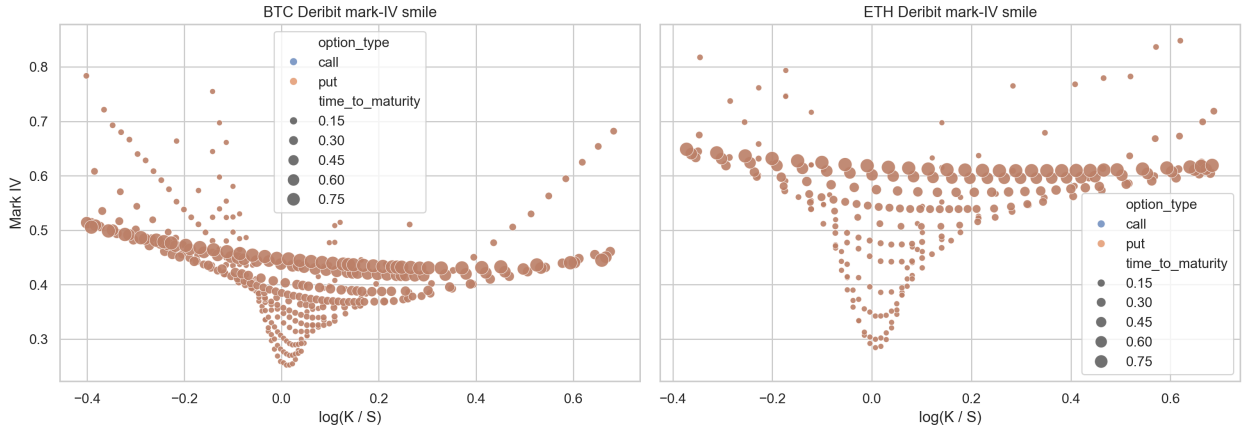

Figure 1: Deribit mark implied-volatility smiles across strikes and maturities.

Inference. A flat Black-Scholes volatility input would imply a roughly horizontal surface. The observed BTC and ETH smiles are curved and dispersed, indicating that the market prices skew, maturity effects, and non-Gaussian tail risk.

The course's implied-volatility concept is important here. Implied volatility is not simply another volatility estimator; it is the volatility that forces the Black-Scholes formula to match a traded option price. A smile therefore means the market is using Black-Scholes as a quote convention while rejecting its constant-volatility distributional assumption. Crypto option studies document precisely this pattern in Bitcoin options [12, 3]. This is the bridge from the data section to the model-innovation section.

Comparative lens. The same smile carries different information for each model. Black-Scholes treats it as model error; Merton and dynamic jump intensity read it as evidence of discontinuous tail risk; MR-ISVM reads it as a state-dependent surface; GARCH reads it as volatility clustering; and SVCJ reads it as joint spot-volatility stress.

3 Evaluation Metrics and Comparative Protocol

To keep the candidate comparison at the same level throughout the report, every model is evaluated on the same cleaned option universe. For option i, let V_i^{mkt} denote the Deribit USD market price and $\widehat{V}_i^{(m)}$ denote the price produced by model m. The model pricing error is:

$$e_i^{(m)} = \widehat{V}_i^{(m)} - V_i^{mkt}.$$

The key cross-sectional pricing metrics are:

$$\text{Bias}^{(m)} = \frac{1}{N}\sum_{i=1}^{N} e_i^{(m)},$$

$$\text{MSE}^{(m)} = \frac{1}{N}\sum_{i=1}^{N}\left(e_i^{(m)}\right)^2, \qquad \text{RMSE}^{(m)} = \sqrt{\text{MSE}^{(m)}},$$

$$\text{MAE}^{(m)} = \frac{1}{N}\sum_{i=1}^{N}\left|e_i^{(m)}\right|, \qquad \text{MedAE}^{(m)} = \text{median}_i\left(\left|e_i^{(m)}\right|\right),$$

$$\text{P90AE}^{(m)} = Q_{0.90}\left(\left|e_i^{(m)}\right|\right).$$

MAE is the main headline metric because it is in USD units and is less dominated by a small number of very expensive contracts than MSE. RMSE is still reported because it penalizes large pricing misses more aggressively.

For pairwise improvement against Black-Scholes, we use:

$$\Delta\text{AE}_i^{(m)} = \left|e_i^{BS}\right| - \left|e_i^{(m)}\right|, \qquad \text{ShareImproved}^{(m)} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{1}\{\Delta\text{AE}_i^{(m)} > 0\}.$$

The paired tests in the calibration section test whether $\Delta\text{AE}_i^{(Merton)}$ is significantly positive.

For standardized crash protection, we price a 30-day 5% out-of-the-money put under each model and define its model premium over Black-Scholes as:

$$\text{CrashPremium}^{(m)} = \widehat{V}^{(m)}_{\text{30d, 5\% OTM put}} - \widehat{V}^{BS}_{\text{30d, 5\% OTM put}}.$$

For hedge simulations, each path p produces a final short-option hedge P&L, denoted $\mathrm{PnL}_p^{(m)}$. We summarize hedging with:

$$\mathrm{MeanAbsHedgeError}^{(m)} = \frac{1}{P}\sum_{p=1}^{P}\left|\mathrm{PnL}_p^{(m)}\right|, \qquad \mathrm{LeftTail}_{5\%}^{(m)} = Q_{0.05}\left(\mathrm{PnL}_p^{(m)}\right).$$

The first metric measures average replication error; the second measures adverse tail loss.

4 Baseline Pricing Framework

4.1 Black-Scholes Benchmark

For European calls and puts with no dividend yield:

$$C = SN(d_1) - Ke^{-rT}N(d_2), \qquad P = Ke^{-rT}N(-d_2) - SN(-d_1),$$

where

$$d_1 = \frac{\log(S/K) + (r + \sigma^2/2)T}{\sigma\sqrt{T}}, \qquad d_2 = d_1 - \sigma\sqrt{T}.$$

Volatility is estimated from log returns, following the course treatment that log returns aggregate naturally across time and that annualized volatility scales with the square root of time. The physical drift is not used directly in risk-neutral pricing because the Black-Scholes PDE and risk-neutral valuation remove μ through replication.

4.2 CRR Binomial Tree

The CRR tree uses

$$u = e^{\sigma\sqrt{\Delta t}}, \qquad d = \frac{1}{u}, \qquad p_Q = \frac{e^{r\Delta t} - d}{u - d}.$$

At each node:

$$V_t = e^{-r\Delta t}\big(p_Q V_u + (1 - p_Q)V_d\big).$$

For American options, the backward step replaces continuation value with

$$V_t = \max\{\text{continuation value},\ \text{immediate exercise value}\}.$$

This matters because the binomial tree is not only a numerical method. It is the discrete version of the course's no-arbitrage argument: p_Q is not the real-world probability that BTC or ETH goes up, but the probability that makes discounted prices consistent with replication.

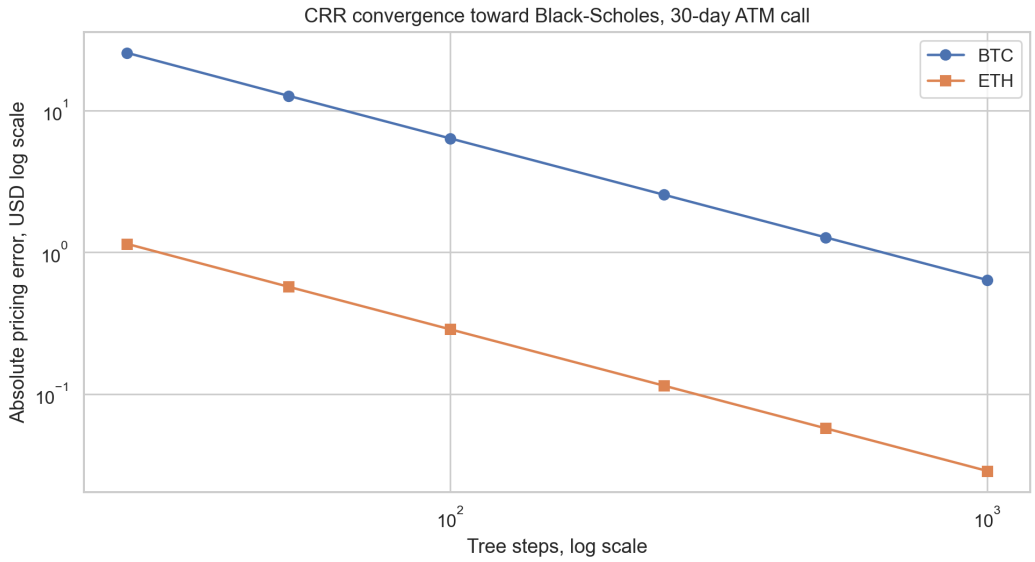

Figure 2: CRR convergence toward the Black-Scholes benchmark for 30-day at-the-money calls.

Table 2: European and American CRR pricing, 30-day at-the-money options

Currency	Option	Spot/Strike	BS Eur.	CRR Eur.	CRR Am.	Am. premium
BTC	call	80803.9	2583.31	2582.03	2582.03	0.00
BTC	put	80803.9	2516.97	2515.69	2519.10	3.41
ETH	call	2263.3	115.55	115.49	115.49	0.00
ETH	put	2263.3	113.69	113.63	113.72	0.09

Inference. The American call premium is effectively zero for non-dividend underlyings, while American puts carry a small early-exercise premium. This is exactly the qualitative result expected from the course theory: early exercise destroys call time value when there is no dividend benefit, but it can be rational for puts when the option is sufficiently in the money.

Comparative role. Black-Scholes and CRR are deliberately kept as the control models. They are not expected to win the crypto smile contest; their role is to anchor no-arbitrage logic, expose how far a diffusion benchmark misses market prices, and make later candidate improvements economically interpretable.

5 Candidate Model Formulations

5.1 Merton as Classical Jump Benchmark

The Merton jump-diffusion model was introduced in 1976 [9]. It is therefore not recent enough to be the final innovation by itself. In this report, it serves as the first serious extension beyond Black-Scholes: a classical benchmark that tests whether explicit price jumps improve crypto option pricing.

The model replaces the diffusion-only assumption with:

$$\frac{dS_t}{S_{t^-}} = (r - \lambda\kappa)dt + \sigma dW_t^Q + (J-1)dN_t,$$

where

$$\log J \sim N(\mu_J, \delta_J^2), \qquad \kappa = E[J-1] = e^{\mu_J + \frac{1}{2}\delta_J^2} - 1.$$

Here λ is the annual jump intensity, μ_J is average log jump size, and δ_J is jump-size volatility. The compensator $\lambda\kappa$ keeps the discounted stock price consistent with risk-neutral valuation.

Conditional on n jumps, terminal log-price remains Gaussian. Hence the Merton option value is a Poisson mixture of conditional Black-Scholes-style prices:

$$V^{Merton}(S,K,T) = \sum_{n=0}^{\infty} e^{-\lambda T}\frac{(\lambda T)^n}{n!} V_n^{BS}(S,K,T).$$

This extension preserves the course's risk-neutral framework while adding a realistic channel for crypto discontinuities. The compensator is essential: without it, the jump term would change the drift of the discounted stock process and break the martingale logic that motivates risk-neutral pricing.

5.2 Candidate A: Stochastic Volatility with Correlated Jumps

The first modern candidate is a stochastic volatility with correlated jumps (SVCJ) model. This model family is closely related to affine jump-diffusion work in option pricing [1, 5], and it has been applied directly to cryptocurrency options in recent empirical work [6]. Its core idea is that crypto crashes are not only spot-price jumps; they are often volatility events as well.

A compact SVCJ specification is:

$$\frac{dS_t}{S_{t^-}} = (r - \lambda\mu_S)dt + \sqrt{v_t}\, dW_t^S + Y_t^S dN_t,$$

$$dv_t = \kappa_v(\theta_v - v_t)dt + \xi\sqrt{v_t}\, dW_t^v + Y_t^v dN_t, \qquad dW_t^S dW_t^v = \rho\, dt.$$

Here v_t is stochastic variance, ρ captures leverage/skew, and (Y_t^S, Y_t^v) allows price jumps and volatility jumps to arrive together. Compared with Merton, SVCJ can explain a more realistic crypto stress state:

$$\text{spot falls sharply} \quad + \quad \text{implied volatility jumps upward.}$$

In the implemented proxy, the historical return series initializes v_0, θ_v, κ_v, variance-jump size, and leverage correlation. The option calibration then refines only the parameters that enter the moment-matched pricing equation directly: the variance multiplier and vol-of-vol ξ. This avoids an unidentified calibration problem: ρ is economically important for skew, but in our reduced moment proxy it is reported as a historical state estimate rather than optimized as if it affected prices.

5.3 Candidate B: Market-Regime Implied Stochastic Volatility

The second candidate is a market-regime implied stochastic volatility model (MR-ISVM), motivated by recent crypto option research [10]. The key idea is that crypto option surfaces behave differently across regimes such as calm markets, trending markets, and stress markets. A reduced formulation is:

$$z_t \in \{1,\dots,M\}, \qquad \sigma_{\mathrm{IV}}(K,T,t) = f_{z_t}(K,T),$$
$$V_t = BS(S_t,K,T,r,\sigma_{\mathrm{IV}}(K,T,t))\,.$$

The regime variable z_t can be estimated from returns, realized volatility, volume, or option-surface features. Compared with Merton, MR-ISVM does not force one set of jump parameters to explain all market states; it allows the implied volatility surface itself to become state-dependent.

Our fitted MR-ISVM surface uses a 12-factor design in log-moneyness $x = \log(K/S)$:

$$\begin{aligned}\sigma_{\mathrm{IV}} = \mathrm{clip}\big(&\beta_0 + \beta_1 x + \beta_2|x| + \beta_3 x^2 + \beta_4\sqrt{T} + \beta_5 T \\ &+ \beta_6 I_{put} + \beta_7 I_{put} x + \beta_8 I_{short} \\ &+ \beta_9 x\sqrt{T} + \beta_{10} I_{put}\sqrt{T} + \beta_{11} x^3,\ \sigma_{\min}(T), \sigma_{\max}\big).\end{aligned}$$

The cross term $x\sqrt{T}$ lets skew vary by maturity, $I_{put}\sqrt{T}$ lets the put wing have a different term slope, and x^3 adds asymmetric wing curvature. This is the main code-level refinement from the teammate merge: MR-ISVM is no longer just a static nine-term smile fit, but a regime-aware and maturity-aware surface benchmark.

5.4 Candidate C: Time-Varying Jump Intensity

The third candidate is a dynamic jump-intensity model. Recent Bitcoin option evidence emphasizes that jump magnitude and jump arrival rates are time-varying [3]. This is important for crypto because large moves can cluster through leverage, liquidations, and margin pressure.

A parsimonious extension is:

$$\frac{dS_t}{S_{t^-}} = (r - \lambda_t \kappa)dt + \sigma dW_t^Q + (J-1)dN_t,$$

$$d\lambda_t = \beta(\bar{\lambda} - \lambda_t)dt + \alpha\, dN_t.$$

After a jump, λ_t increases, then mean-reverts. Compared with Merton's constant λ, this model captures jump clustering and liquidation-cascade risk.

5.5 Candidate D: GARCH Option Pricing

The fourth candidate is a GARCH option-pricing model, which has also been studied for Bitcoin options [11]. It is a discrete-time alternative that connects naturally to the course's historical volatility estimation section:

$$r_{t+1} = \mu + \sqrt{h_{t+1}}\varepsilon_{t+1},$$
$$h_{t+1} = \omega + \alpha r_t^2 + \beta h_t.$$

Option prices can then be computed by simulation under a risk-neutralized conditional variance process. Compared with Merton, GARCH does not require discontinuous jumps, but it captures volatility clustering. It is therefore useful as a competing explanation for smiles: are crypto smiles mainly jump risk, or time-varying volatility?

Table 3: Equal-level implementation protocol for all candidate models

Model	Calibration object	Pricing engine	Same-level diagnostic
Black-Scholes	Historical log-return variance	Closed-form diffusion formula	Full-universe MAE/RMSE and Greeks
Merton	Weighted Deribit option-price error	Poisson mixture of Black-Scholes prices	Full-universe MAE/RMSE, jump sensitivity, hedging
SVCJ proxy	Historical variance state plus identifiable variance and ξ calibration	Moment-matched stochastic variance with Merton jumps	Full-universe MAE/RMSE and standardized crash put
MR-ISVM	Regime-aware Deribit mark-IV cross section	Black-Scholes with active-regime fitted IV surface	Full-universe MAE/RMSE and standardized crash put
Dynamic jump	Weighted option-price error with mean-reverting λ_t	Merton mixture with horizon-dependent effective intensity	Full-universe MAE/RMSE and standardized crash put
GARCH	Historical conditional return variance	Black-Scholes with forecast average variance	Full-universe MAE/RMSE and standardized crash put

This protocol is deliberately symmetric. Each candidate uses the same cleaned Deribit universe, the same 80-contract calibration discipline where option calibration is needed, and the same diagnostic definitions from the previous section. The SVCJ implementation is labelled as a proxy because a

full affine SVCJ calibration normally requires a time series of option surfaces; here we use a moment-matched variance state so it can be compared fairly with the available live snapshot. The empirical comparison is then carried through the distribution, calibration, sensitivity, Greeks, and hedging sections rather than isolated here.

Table 4: Regime-aware MR-ISVM surface diagnostics

Currency	Active regime	Calibrated regimes	Coefficients	Rel. price RMSE	IV floor/cap
BTC	calm	calm, stress	12	0.2610	0.20 / 3.00
ETH	trend	calm, trend	12	0.2945	0.20 / 3.00

Model-design inference. BTC is classified as a calm active regime but still has a calibrated stress fallback; ETH is classified as trend with a calm fallback. This is exactly why the surface model is useful in a course project: the Black-Scholes formula remains the pricing map, while the implied volatility input becomes a market-state object rather than a single scalar.

6 Historical Distribution and Jump Diagnostics

We compute one-year BTC/ETH log returns from Deribit index history. Jumps are detected using a robust z-score threshold of 4.0. Non-jump observations initialize diffusive volatility; jump observations initialize jump intensity and jump-size distribution.

Figure 3: Robust jump detection in BTC and ETH index log returns.

Table 5: Historical jump initialization

Currency	Obs.	Jumps	Diff. σ	λ	μ_J	δ_J	Worst	Best
BTC	1459	39	27.62%	39.05	-0.575%	3.162%	-6.240%	4.139%
ETH	1459	40	44.34%	40.05	-0.768%	4.760%	-7.373%	6.014%

Table 6: Return distribution diagnostics

Currency	Ann. mean	Ann. vol	Skew	Ex. kurtosis	JB statistic	JB p-value
BTC	-23.77%	33.70%	-0.556	4.910	1528.21	$< 10^{-12}$
ETH	-11.52%	53.12%	-0.414	3.547	799.51	$< 10^{-12}$

Inference. Jarque-Bera tests strongly reject normality for both BTC and ETH returns. Both assets exhibit negative skew and excess kurtosis, validating the modeling decision to go beyond a lognormal diffusion. ETH has higher annualized volatility and jump-size volatility, while BTC still exhibits material discontinuous downside risk.

This section deepens the Black-Scholes estimation chapter. In the classical model, the statistic we need is the standard deviation of log returns. For BTC/ETH, that statistic is still useful, but it is incomplete: skewness, kurtosis, and jump frequency explain why a single σ cannot reproduce the market smile. The empirical distribution therefore motivates a pricing model with both a continuous volatility channel and a discontinuous jump channel.

Comparative lens. These diagnostics initialize different model states. Black-Scholes uses only diffusive volatility; Merton uses diffusive volatility plus jump intensity and jump-size moments; dynamic jump intensity asks whether λ_t should vary across horizons; GARCH uses conditional variance persistence; SVCJ uses both variance state and variance-jump state; and MR-ISVM uses the option surface itself to infer the current regime.

7 Market Calibration and Statistical Evidence

Merton parameters are calibrated to liquid Deribit option marks by minimizing weighted relative pricing error. The calibration uses 80 liquid options per asset; diagnostics are computed on the full cleaned universe. Conceptually, this is an implied-parameter exercise: just as the course defines implied volatility by inverting Black-Scholes, we infer jump parameters by matching a cross-section of market option prices.

Table 7: Black-Scholes versus calibrated Merton pricing error

Currency	Cal. σ	Cal. λ	Cal. μ_J	Cal. δ_J	BS MAE	Merton MAE
BTC	14.94%	39.05	-3.166%	5.617%	1292.78	401.73
ETH	26.90%	40.05	-3.185%	6.861%	36.19	18.72

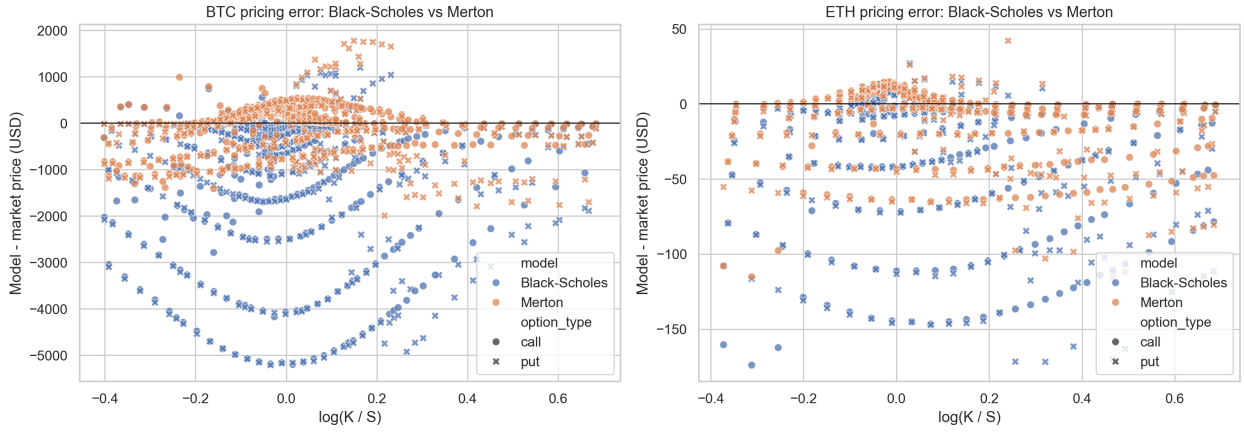

Figure 4: Model pricing errors against Deribit option marks.

Table 8: Paired statistical tests for absolute-error reduction

Currency	Pairs	Mean impr.	Median impr.	Share impr.	t-test p	Wilcoxon p
BTC	748	891.04	236.53	78.88%	1.39×10^{-65}	6.46×10^{-73}
ETH	516	17.46	2.06	70.93%	1.19×10^{-44}	9.60×10^{-32}

Table 9: Full candidate pricing comparison on the cleaned option universe

Currency	Model	Options	Bias	MAE	RMSE
BTC	Merton	748	-109.94	401.73	560.05
BTC	Dynamic jump	748	-176.11	425.22	603.56
BTC	SVCJ proxy	748	425.89	598.22	810.23
BTC	MR-ISVM surface	748	-260.56	639.04	1211.78
BTC	GARCH variance	748	-683.01	763.75	1115.74
BTC	Black-Scholes	748	-1229.03	1292.78	1939.14
ETH	MR-ISVM surface	516	-1.82	14.63	28.98
ETH	GARCH variance	516	-14.38	17.92	29.10
ETH	Merton	516	-13.11	18.72	29.45
ETH	Dynamic jump	516	-14.27	19.62	30.90
ETH	SVCJ proxy	516	28.23	30.66	45.08
ETH	Black-Scholes	516	-34.08	36.19	57.70

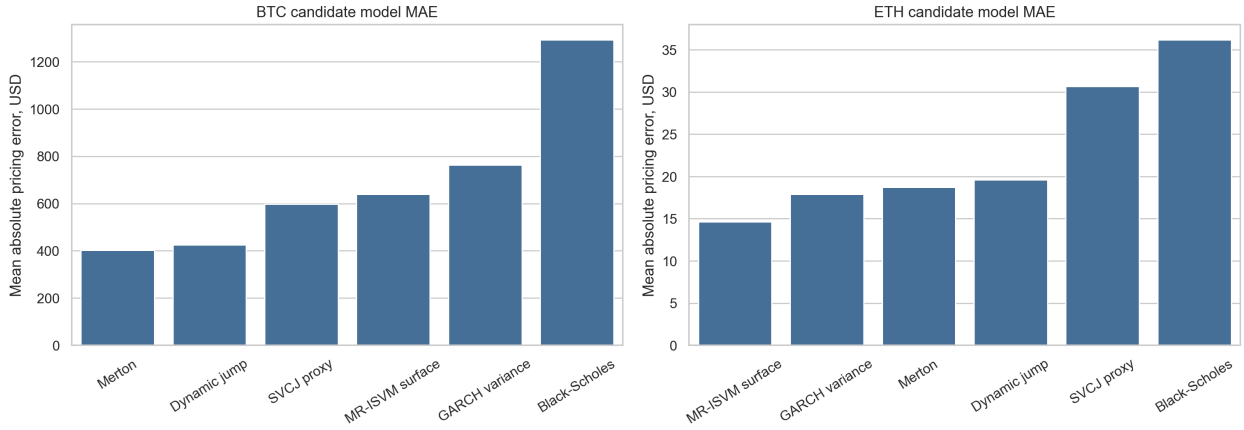

Figure 5: Equal-level candidate comparison using mean absolute pricing error.

Inference. The Merton improvement is not merely visual. Paired t-tests, Wilcoxon signed-rank tests, and sign tests all show overwhelming evidence that the jump-diffusion model reduces absolute pricing error relative to the Black-Scholes historical-volatility baseline. In this snapshot, Merton reduces MAE from USD 1292.78 to USD 401.73 for BTC and from USD 36.19 to USD 18.72 for ETH.

The paired design is important. Each option is priced twice, once by Black-Scholes and once by Merton, so the test compares models instrument by instrument rather than comparing two unrelated samples. This mirrors the course's emphasis on using a benchmark model first, then measuring model error against market prices.

Comparative lens. The broader candidate table changes the interpretation. Merton gives the best BTC MAE because the BTC cross section in this snapshot rewards a structural downside-jump premium. MR-ISVM gives the best ETH MAE because ETH is classified in a trend regime and the active implied-volatility surface captures the live smile more tightly than a single jump distribution. Dynamic jump intensity is close to Merton but does not dominate it. GARCH improves over Black-Scholes by capturing volatility persistence, but it remains weaker than explicit jump or surface

models. The SVCJ proxy prices tail risk aggressively, which is useful for stress analysis but still too conservative for average cross-sectional marking without a full option-surface time series.

Table 10: Selected calibrated states for the modern candidates

Model	BTC fitted state	ETH fitted state
Merton	$\sigma = 0.1494$, $\lambda = 39.05$, $\mu_J = -0.0317$, $\delta_J = 0.0562$	$\sigma = 0.2690$, $\lambda = 40.05$, $\mu_J = -0.0319$, $\delta_J = 0.0686$
SVCJ proxy	$v_0 = 0.0608$, $\theta = 0.0223$, $\xi = 0.2870$, multiplier $= 0.306$	$v_0 = 0.0818$, $\theta = 0.0724$, $\xi = 0.3529$, multiplier $= 0.343$
MR-ISVM	active calm, two regimes, 12 coefficients, RMSE $= 0.2610$	active trend, two regimes, 12 coefficients, RMSE $= 0.2945$
Dynamic jump	$\sigma = 0.2032$, $\bar{\lambda} = 32.61$, $\lambda_t = 32.61$, $\beta = 1.60$	$\sigma = 0.3211$, $\bar{\lambda} = 33.32$, $\lambda_t = 33.33$, $\beta = 0.83$
GARCH variance	$\alpha + \beta = 0.980$	$\alpha + \beta = 0.959$

8 Interest Rate Sensitivity

We price 30-day at-the-money calls and puts under $r = 1\%$ and $r = 5\%$ to isolate interest-rate sensitivity.

Table 11: Interest-rate sensitivity, 30-day at-the-money options

Currency	Option	Rate	BS price	Merton price
BTC	call	1%	2583.31	3872.19
BTC	call	5%	2714.82	4011.44
BTC	put	1%	2516.97	3805.85
BTC	put	5%	2383.66	3680.28
ETH	call	1%	115.55	138.55
ETH	call	5%	119.12	142.29
ETH	put	1%	113.69	136.69
ETH	put	5%	109.85	133.02

Inference. The rate effect has the expected sign from put-call parity and discounting: higher rates increase calls and reduce puts. Over a 30-day crypto horizon, however, jump and volatility assumptions dominate rate sensitivity. This is a useful hierarchy of risks: for short-dated BTC/ETH options, the main modeling battle is not the risk-free rate, but the distribution of the underlying.

Comparative lens. The same hierarchy applies to the candidate models. Interest rates are a shared input, while the dominant disagreement across Black-Scholes, Merton, MR-ISVM, SVCJ, dynamic jump intensity, and GARCH comes from the assumed distribution of S_T. This is why we evaluate both average pricing error and standardized crash-protection values.

9 Jump-Parameter Sensitivity

We stress the calibrated Merton model by repricing 30-day, 5% out-of-the-money puts under different jump-intensity and jump-volatility multipliers. This directly measures the value of crash protection.

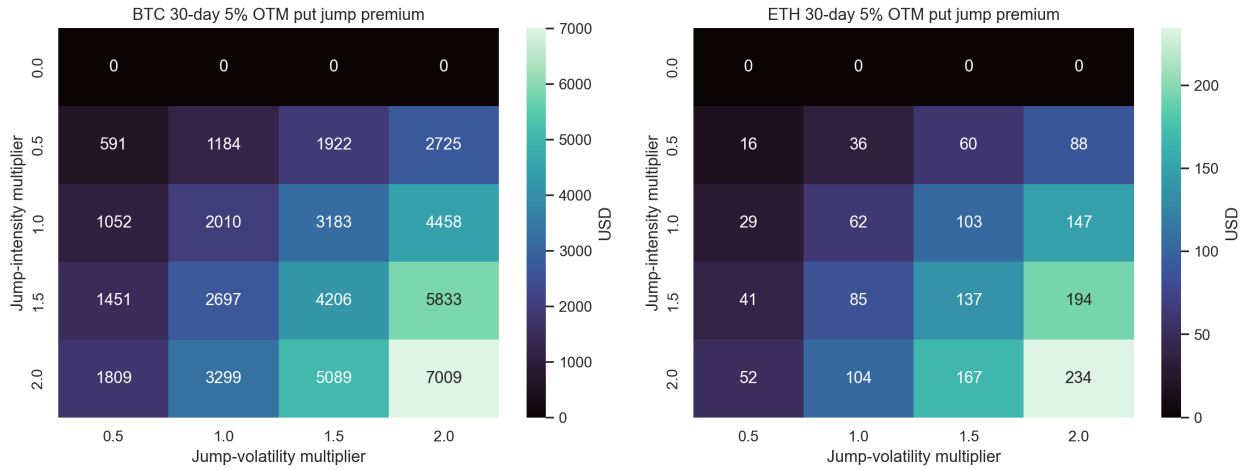

Figure 6: Sensitivity of 30-day 5% OTM put jump premium to jump intensity and jump-size volatility.

Inference. The jump premium rises nonlinearly as either jump intensity or jump-size volatility increases. For BTC, doubling both calibrated jump intensity and jump volatility increases the 30-day 5% OTM put premium by about USD 7009 relative to the diffusion-only baseline. For ETH, the comparable premium is about USD 234. This is a compact risk-management result: crash protection is primarily a jump-risk instrument, not just a high-volatility instrument.

The Risk Management chapter brings this sensitivity analysis into model-risk language. Greeks measure local sensitivities, but a jump parameter shock is a structural stress test. The heatmap therefore complements Delta and Vega: it asks how much the option value changes when the assumed tail process itself becomes more severe.

Table 12: Standardized 30-day 5% OTM put values across candidates

Currency	Model	Put value	Premium over BS
BTC	Black-Scholes	954.44	0.00
BTC	Merton	2192.77	1238.34
BTC	SVCJ proxy	2401.30	1446.86
BTC	MR-ISVM surface	1858.37	903.93
BTC	Dynamic jump	2109.09	1154.65
BTC	GARCH variance	1381.66	427.23
ETH	Black-Scholes	63.54	0.00
ETH	Merton	87.56	24.02
ETH	SVCJ proxy	102.82	39.28
ETH	MR-ISVM surface	73.00	9.46
ETH	Dynamic jump	85.86	22.33
ETH	GARCH variance	80.77	17.24

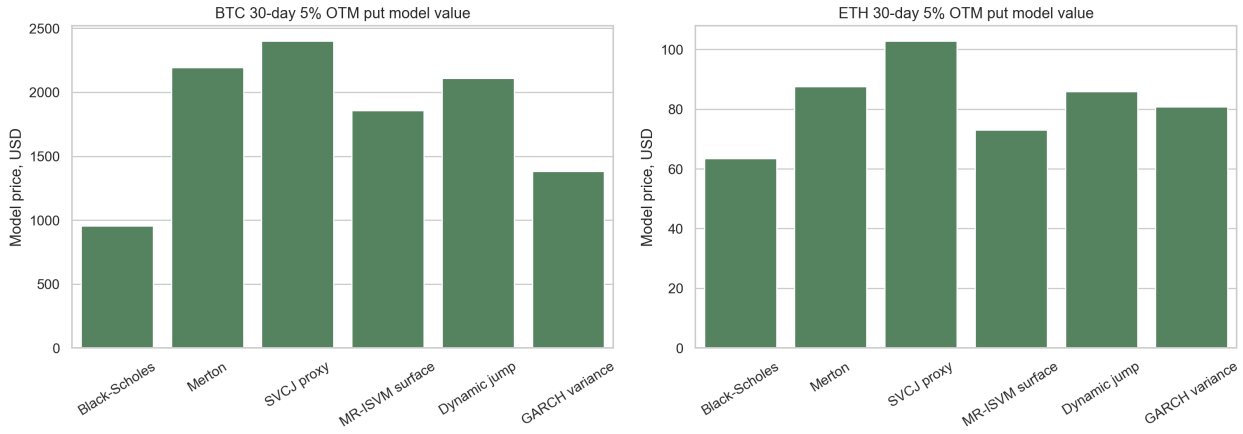

Figure 7: Model-implied crash-protection values for standardized 30-day 5% OTM puts.

Comparative lens. The ranking changes when the objective is crash protection rather than average pricing error. SVCJ produces the largest 30-day downside protection value because it prices the joint event "spot falls and volatility jumps." Merton and dynamic jump intensity also attach large value to discontinuous downside moves. GARCH is more conservative because volatility clustering alone does not create jump losses. MR-ISVM sits between these cases: it reads the current market smile, but it does not explicitly simulate jump arrival.

10 Greeks and Risk Management

In the course framework, Greeks are sensitivities of option value to state variables and parameters. We compute Black-Scholes Greeks analytically and Merton Greeks by finite differences.

Table 13: Greeks for 30-day at-the-money options at $r = 1\%$

Currency	Option	BS Δ	Merton Δ	BS Γ	Merton Γ	BS Vega	Merton Vega
BTC	call	0.5199	0.5679	0.0000623	0.0000410	9227.12	3282.71
BTC	put	-0.4801	-0.4321	0.0000623	0.0000410	9227.12	3282.71
ETH	call	0.5279	0.5608	0.0013837	0.0011699	258.14	132.43
ETH	put	-0.4721	-0.4392	0.0013837	0.0011699	258.14	132.43

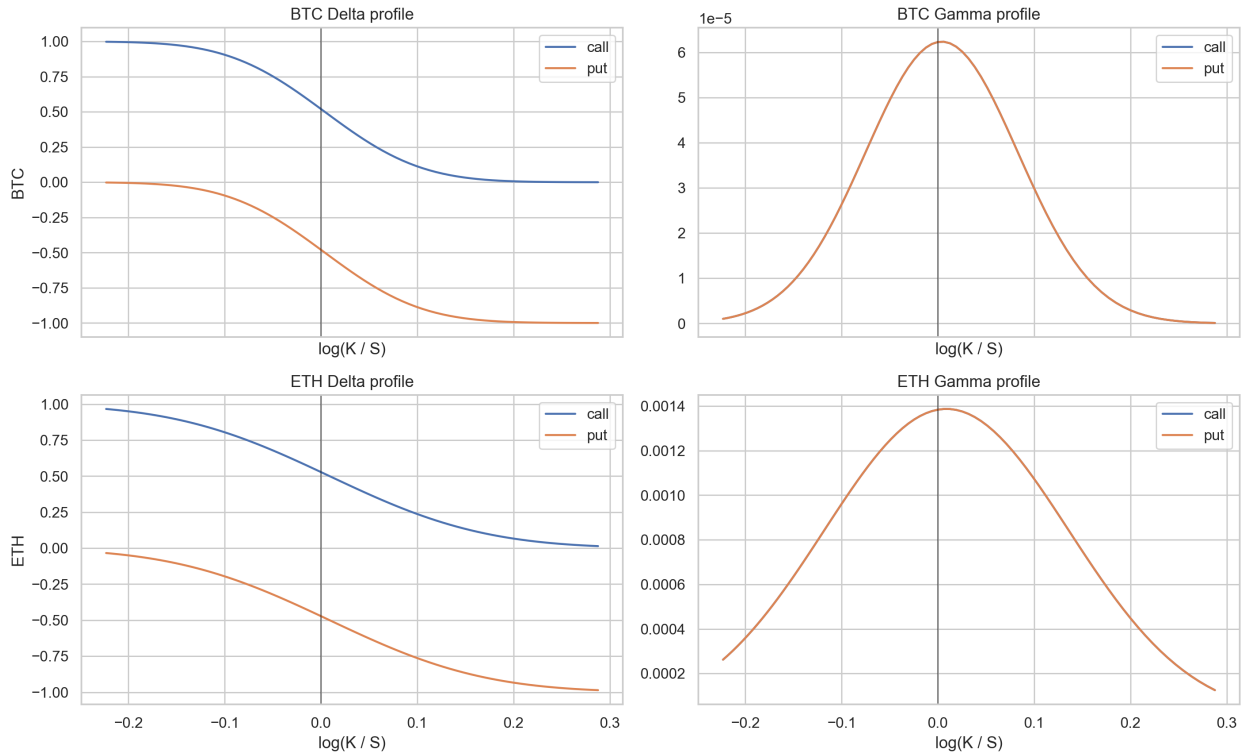

Figure 8: Black-Scholes Delta and Gamma profiles across log-moneyness.

Inference. Merton deltas differ from Black-Scholes deltas because downside jump risk shifts the risk-neutral distribution. Gamma and Vega are also reallocated: some tail value is explained by jump parameters rather than purely by diffusive volatility. A desk hedging only Black-Scholes delta would therefore remain exposed to jump convexity.

The course relation

$$\Theta + rS\Delta + \frac{1}{2}\sigma^2 S^2 \Gamma = r\Pi$$

is a useful interpretation device. Under pure diffusion, Delta and Gamma summarize the first two local spot risks. Under jumps, however, a finite move in S can dominate the Taylor approximation. This is why the Greeks section must be read together with the jump-sensitivity and hedge-simulation sections.

Comparative lens. Greeks are local diagnostics, while candidate-model differences are partly global distributional assumptions. Black-Scholes and GARCH mainly modify the volatility channel; Merton and dynamic jump intensity modify the jump-arrival channel; SVCJ modifies both variance and jump channels; MR-ISVM embeds the market's current state into the implied-volatility surface. The practical implication is that a desk should not interpret Delta or Vega without also checking which model generated the distribution around those Greeks.

11 Jump-Hedging Simulation

Continuous delta hedging is an idealization. The Risk Management chapter notes that periodic hedging introduces replication error. We simulate 30-day, 5% OTM put hedging under Merton jump paths.

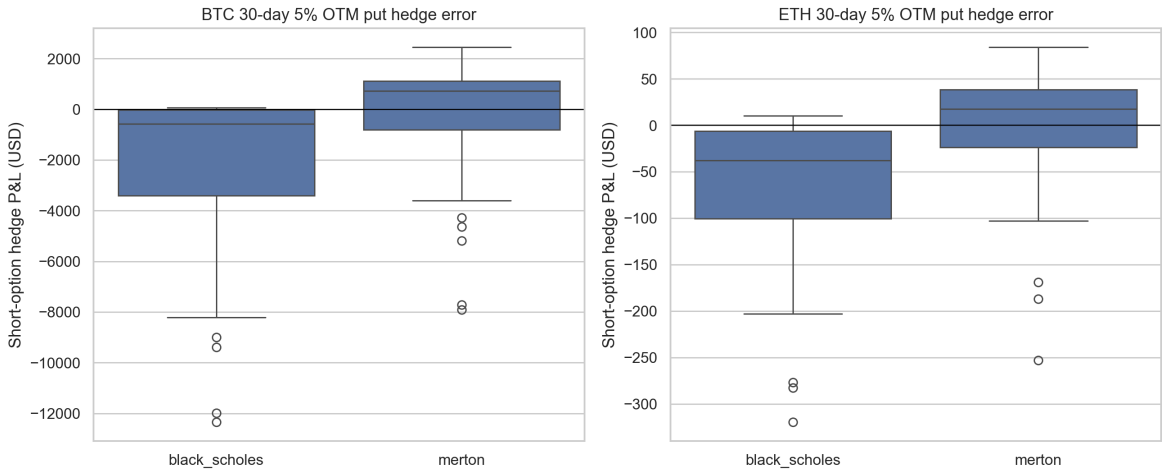

Figure 9: Discrete hedge P&L under Merton jump paths.

Table 14: Discrete hedge-error summary

Currency	Model	Mean P&L	Median P&L	Std P&L	Mean |P&L|	5% P&L
BTC	Black-Scholes	-2025.62	-566.49	2798.69	2031.84	-7696.61
BTC	Merton	-32.78	733.95	1910.57	1399.26	-3643.47
ETH	Black-Scholes	-61.15	-37.81	70.74	62.21	-196.83
ETH	Merton	1.09	17.85	59.91	45.16	-102.30

Inference. Merton-aware hedging reduces mean absolute hedge error for both assets and materially reduces the BTC left-tail hedge loss. It is not perfect: jumps cannot be hedged after they arrive. But it is more honest about the risk process than a diffusion-only hedge.

This result connects directly to the replication logic in the binomial and Black-Scholes chapters. Replication is exact only under the assumed dynamics and rebalancing frequency. Once the true path contains discontinuities, a self-financing delta hedge accumulates residual error. The practical question is therefore not whether hedging eliminates risk, but which model leaves the smaller residual under realistic crypto paths.

Comparative lens. We run path-level hedge simulations for Black-Scholes and Merton because both provide explicit, transparent dynamics for S_t. The other candidates still matter in this section: MR-ISVM is better suited for marking the current surface, SVCJ highlights volatility-jump hedge risk, dynamic jump intensity highlights clustered jump exposure, and GARCH highlights volatility persistence. Together, the comparison separates three desk questions: which model marks best, which model stresses tail protection most, and which model gives a hedgeable path process.

12 Discussion

Black-Scholes is internally coherent under frictionless trading, continuous hedging, constant volatility, and lognormal diffusion. Crypto markets violate these assumptions in several ways:

- returns are heavy-tailed and negatively skewed;
- volatility is regime-dependent;
- investors pay for downside crash protection;
- delta hedging is discrete and liquidity can vanish during stress;

- leverage and liquidation effects can amplify short-horizon moves.

The Merton model is therefore not an ornamental extension. It is a mathematically controlled way to carry the course's risk-neutral framework into the empirical reality of BTC/ETH options. But the equal-level candidate comparison also shows why Merton should not be the final word. Merton currently gives the cleanest BTC cross-sectional MAE; MR-ISVM gives the cleanest ETH cross-sectional MAE; SVCJ is more explicit about volatility jumps and dominates in standardized crash-premium analysis; dynamic intensity targets jump clustering; and GARCH isolates volatility persistence as a competing explanation.

The sequence of chapters brings the analysis to a layered conclusion: risk-neutral pricing tells us how to value claims once dynamics are specified; binomial trees show how replication works discretely; Black-Scholes gives the diffusion benchmark; implied volatility reveals the benchmark's market misfit; and risk management exposes the hedging cost of that misfit. The model-comparison layer is our response to that chain of evidence.

13 Conclusion

This project implements a complete option pricing workflow: Black-Scholes prices and Greeks, CRR European and American trees, implied-volatility smiles, market calibration, candidate model comparison, and dynamic hedge simulation.

The innovation is no longer only the Merton jump-diffusion layer. It is the equal-level comparison of five pricing views: diffusion, constant jump intensity, stochastic volatility jumps, regime implied volatility, dynamic jump intensity, and GARCH conditional variance.

The central finding is that crypto option markets price jump risk and volatility-regime risk explicitly. Black-Scholes should be treated as a control model and teaching benchmark. A professional crypto options desk would not select one model for every purpose: it would use the best surface or structural model for marking, then retain jump and SVCJ-style engines for stress testing, Greeks, and risk management.

References

[1] Bates, D. S. (1996). Jumps and stochastic volatility: Exchange rate processes implicit in Deutsche Mark options. *The Review of Financial Studies*, 9(1), 69–107.

[2] Black, F., and Scholes, M. (1973). The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3), 637–654.

[3] Chen, K. S., and Yang, J. J. (2024). Price dynamics and volatility jumps in bitcoin options. *Financial Innovation*, 10, Article 132.

[4] Cox, J. C., Ross, S. A., and Rubinstein, M. (1979). Option pricing: A simplified approach. *Journal of Financial Economics*, 7(3), 229–263.

[5] Duffie, D., Pan, J., and Singleton, K. (2000). Transform analysis and asset pricing for affine jump-diffusions. *Econometrica*, 68(6), 1343–1376.

[6] Hou, A. J., Wang, W. N., Chen, C. Y.-H., and Härdle, W. K. (2020). Pricing cryptocurrency options. *Journal of Financial Econometrics*, 18(2), 250–279.

[7] Jiang, W. (2026). IEDA4331 Financial Engineering lecture notes: risk-neutral pricing, binomial trees, Black-Scholes-Merton, and risk management. Hong Kong University of Science and Technology.

[8] Merton, R. C. (1973). Theory of rational option pricing. *The Bell Journal of Economics and Management Science*, 4(1), 141–183.

[9] Merton, R. C. (1976). Option pricing when underlying stock returns are discontinuous. *Journal of Financial Economics*, 3(1–2), 125–144.

[10] Saef, D., Wang, Y., and Aste, T. (2022). Regime-based implied stochastic volatility model for crypto option pricing. arXiv:2208.12614.

[11] Venter, P. J., and Maré, E. (2021). Univariate and multivariate GARCH models applied to Bitcoin futures option pricing. *Journal of Risk and Financial Management*, 14(6), Article 261.

[12] Zulfiqar, N., and Gulzar, S. (2021). Implied volatility estimation of Bitcoin options and the stylized facts of option pricing. *Financial Innovation*, 7, Article 67.

A Implementation Notes

The implementation is modular:

- `src/data_deribit.py`: public Deribit data access and caching.
- `src/preprocessing.py`: quote cleaning, USD conversion, returns, and jump detection.
- `src/black_scholes.py`, `src/binomial.py`, and `src/merton_jump.py`: baseline and jump pricing engines.
- `src/candidate_models.py`: comparable modern candidates.
- `src/calibration.py` and `src/risk.py`: calibration and hedge simulation.
- `scripts/generate_report_assets.py`: reproducible report tables and figures.

B Core Programming Code Listings

This appendix records the key programming modules that implement the report's pricing, calibration, candidate-model, and risk-management analysis. The listings are selected source excerpts rather than every helper function in the repository; each one is labelled so the implementation can be traced directly to the mathematical sections of the report.

```python
    def get_json(
        self,
        method: str,
        params: Mapping[str, Any] | None = None,
        *,
        use_cache: bool = True,
    ) -> dict[str, Any]:
        """Call a public Deribit method and return the decoded JSON payload."""

        params = {k: v for k, v in (params or {}).items() if v is not None}
        cache_path = self._cache_path(method, params)

        if use_cache and cache_path and cache_path.exists():
            age = time.time() - cache_path.stat().st_mtime
            if age <= self.ttl_seconds:
                return json.loads(cache_path.read_text())

        url = f"{self.base_url.rstrip('/')}/{method}"
        if requests is not None:
            response = requests.get(url, params=params, timeout=self.timeout)
            response.raise_for_status()
            payload = response.json()
        else:  # pragma: no cover
            query = urllib.parse.urlencode(params)
            request_url = f"{url}?{query}" if query else url
            try:
                with urllib.request.urlopen(request_url, timeout=self.timeout) as response:
                    payload = json.loads(response.read().decode("utf-8"))
            except urllib.error.URLError as exc:
                raise DeribitAPIError(f"Deribit request failed: {exc}") from exc

        if "error" in payload:
            raise DeribitAPIError(str(payload["error"]))
        if "result" not in payload:
            raise DeribitAPIError(f"Unexpected Deribit response: {payload}")

        if use_cache and cache_path:
            cache_path.parent.mkdir(parents=True, exist_ok=True)
            cache_path.write_text(json.dumps(payload, indent=2, sort_keys=True))

        return payload
```

Listing 1: Data client for Deribit public market-data requests.

```python
def normalize_option_chain(
    book: pd.DataFrame,
    instruments: pd.DataFrame,
    *,
    valuation_time: pd.Timestamp | None = None,
) -> pd.DataFrame:
    """Merge Deribit book summaries with instrument metadata and add features."""

    if book.empty:
        return pd.DataFrame()

    valuation_time = valuation_time or pd.Timestamp.utcnow()
    valuation_ms = int(valuation_time.timestamp() * 1000)
    instrument_cols = [
        col
        for col in [
            "instrument_name",
            "strike",
            "option_type",
            "expiration_timestamp",
            "contract_size",
            "settlement_currency",
```

```
            "quote_currency",
        ]
        if col in instruments.columns
    ]
    df = book.copy()
    if instrument_cols:
        df = df.merge(instruments[instrument_cols], on="instrument_name", how="left")

    parsed = df["instrument_name"].map(_parse_instrument_name)
    parsed_strike = parsed.map(lambda item: item[0])
    parsed_type = parsed.map(lambda item: item[1])
    df["strike"] = pd.to_numeric(df.get("strike", parsed_strike), errors="coerce").fillna(parsed_strike)
    df["option_type"] = df.get("option_type", parsed_type).fillna(parsed_type).str.lower()
    if "expiration_timestamp" not in df:
        df["expiration_timestamp"] = np.nan

    numeric_cols = [
        "bid_price",
        "ask_price",
        "mid_price",
        "mark_price",
        "underlying_price",
        "interest_rate",
        "open_interest",
        "volume",
        "mark_iv",
        "expiration_timestamp",
    ]
    for col in numeric_cols:
        if col in df:
            df[col] = pd.to_numeric(df[col], errors="coerce")

    df["valuation_timestamp"] = valuation_ms
    df["valuation_datetime"] = pd.to_datetime(valuation_ms, unit="ms", utc=True)
    df["expiration_datetime"] = pd.to_datetime(df["expiration_timestamp"], unit="ms", utc=True)
    df["time_to_maturity"] = (df["expiration_timestamp"] - valuation_ms) / MS_PER_YEAR
    df["rate"] = pd.Series(_normalise_percent(df.get("interest_rate", 0.0)), index=df.index).fillna(0.0)
    df["mark_iv_decimal"] = pd.Series(_normalise_percent(df.get("mark_iv", np.nan)), index=df.index)

    for col in ["bid_price", "ask_price", "mid_price", "mark_price"]:
        if col in df:
            df[f"{col}_usd"] = df[col] * df["underlying_price"]

    mid = df.get("mid_price")
    if mid is None:
        mid = (df["bid_price"] + df["ask_price"]) / 2.0
    derived_mid = (df["bid_price"] + df["ask_price"]) / 2.0
    df["market_price_coin"] = np.where(mid > 0.0, mid, df["mark_price"])
    df["market_price_coin"] = np.where(df["market_price_coin"] > 0.0, df["market_price_coin"], derived_mid)
    df["market_price_usd"] = df["market_price_coin"] * df["underlying_price"]
    df["spread_coin"] = df["ask_price"] - df["bid_price"]
    df["spread_usd"] = df["spread_coin"] * df["underlying_price"]
    df["moneyness"] = df["underlying_price"] / df["strike"]
    df["log_moneyness"] = np.log(df["strike"] / df["underlying_price"])
    df["liquidity_score"] = df[["open_interest", "volume"]].fillna(0.0).sum(axis=1)
    return df.sort_values(["expiration_timestamp", "strike", "option_type"]).reset_index(drop=True)
```

Listing 2: Option-chain normalization: metadata merge, USD conversion, maturity, moneyness, and IV features.

```
def compute_log_returns(index_history: pd.DataFrame) -> pd.DataFrame:
    """Compute timestamped log returns from a Deribit index history frame."""

    df = index_history.sort_values("timestamp").copy()
    df["log_price"] = np.log(df["price"])
    df["log_return"] = df["log_price"].diff()
    return df.dropna(subset=["log_return"]).reset_index(drop=True)


def detect_jumps(
    returns: pd.DataFrame,
    *,
    z_threshold: float = 3.0,
    return_col: str = "log_return",
) -> pd.DataFrame:
```

```
    """Flag large absolute return shocks using a robust z-score."""

    df = returns.copy()
    median = df[return_col].median()
    mad = (df[return_col] - median).abs().median()
    robust_sigma = 1.4826 * mad if mad > 0.0 else df[return_col].std(ddof=1)
    df["jump_z"] = (df[return_col] - median) / max(robust_sigma, 1e-12)
    df["is_jump"] = df["jump_z"].abs() >= z_threshold
    return df
```

Listing 3: Historical return construction and robust jump detection.

```
def d1_d2(S: Any, K: Any, T: Any, r: Any, sigma: Any, q: Any = 0.0):
    """Return Black-Scholes ``d1`` and ``d2`` arrays."""

    S, K, T, r, sigma, q = _broadcast_inputs(S, K, T, r, sigma, q)
    sqrt_T = np.sqrt(np.maximum(T, 1e-16))
    vol_sqrt_T = np.maximum(sigma * sqrt_T, 1e-16)
    d1 = (np.log(S / K) + (r - q + 0.5 * sigma * sigma) * T) / vol_sqrt_T
    d2 = d1 - vol_sqrt_T
    return d1, d2


def bs_price(
    S: Any,
    K: Any,
    T: Any,
    r: Any,
    sigma: Any,
    option_type: Any = "call",
    q: Any = 0.0,
) -> float | np.ndarray:
    """Black-Scholes price with optional continuous carry/dividend yield ``q``."""

    S, K, T, r, sigma, q = _broadcast_inputs(S, K, T, r, sigma, q)
    calls = np.broadcast_to(_is_call(option_type), S.shape)
    call_intrinsic = np.maximum(S - K, 0.0)
    put_intrinsic = np.maximum(K - S, 0.0)
    intrinsic = np.where(calls, call_intrinsic, put_intrinsic)

    valid = (S > 0.0) & (K > 0.0) & (T > 0.0) & (sigma > 0.0)
    out = intrinsic.astype(float)
    if np.any(valid):
        d1, d2 = d1_d2(S[valid], K[valid], T[valid], r[valid], sigma[valid], q[valid])
        discounted_spot = S[valid] * np.exp(-q[valid] * T[valid])
        discounted_strike = K[valid] * np.exp(-r[valid] * T[valid])
        call = discounted_spot * _normal_cdf(d1) - discounted_strike * _normal_cdf(d2)
        put = discounted_strike * _normal_cdf(-d2) - discounted_spot * _normal_cdf(-d1)
        out[valid] = np.where(calls[valid], call, put)
    return _maybe_scalar(out)


def bs_greeks(
    S: Any,
    K: Any,
    T: Any,
    r: Any,
    sigma: Any,
    option_type: Any = "call",
    q: Any = 0.0,
) -> dict[str, float | np.ndarray]:
    """Return Delta, Gamma, Vega, Theta, and Rho.

    Vega and Rho are reported per unit change in volatility/rate, not per
    one-percentage-point change.
    """

    S, K, T, r, sigma, q = _broadcast_inputs(S, K, T, r, sigma, q)
    calls = np.broadcast_to(_is_call(option_type), S.shape)
    valid = (S > 0.0) & (K > 0.0) & (T > 0.0) & (sigma > 0.0)

    delta = np.where(calls, (S > K).astype(float), -(S < K).astype(float))
    gamma = np.full(S.shape, np.nan)
    vega = np.full(S.shape, np.nan)
    theta = np.full(S.shape, np.nan)
    rho = np.full(S.shape, np.nan)
```

```

    if np.any(valid):
        d1, d2 = d1_d2(S[valid], K[valid], T[valid], r[valid], sigma[valid], q[valid])
        sqrt_T = np.sqrt(T[valid])
        exp_q = np.exp(-q[valid] * T[valid])
        exp_r = np.exp(-r[valid] * T[valid])
        pdf_d1 = _normal_pdf(d1)
        nd1 = _normal_cdf(d1)
        nd2 = _normal_cdf(d2)
        nmd1 = _normal_cdf(-d1)
        nmd2 = _normal_cdf(-d2)

        delta_call = exp_q * nd1
        delta_put = exp_q * (nd1 - 1.0)
        gamma_v = exp_q * pdf_d1 / (S[valid] * sigma[valid] * sqrt_T)
        vega_v = S[valid] * exp_q * pdf_d1 * sqrt_T

        theta_common = -S[valid] * exp_q * pdf_d1 * sigma[valid] / (2.0 * sqrt_T)
        theta_call = (
            theta_common
            - r[valid] * K[valid] * exp_r * nd2
            + q[valid] * S[valid] * exp_q * nd1
        )
        theta_put = (
            theta_common
            + r[valid] * K[valid] * exp_r * nmd2
            - q[valid] * S[valid] * exp_q * nmd1
        )
        rho_call = K[valid] * T[valid] * exp_r * nd2
        rho_put = -K[valid] * T[valid] * exp_r * nmd2

        delta[valid] = np.where(calls[valid], delta_call, delta_put)
        gamma[valid] = gamma_v
        vega[valid] = vega_v
        theta[valid] = np.where(calls[valid], theta_call, theta_put)
        rho[valid] = np.where(calls[valid], rho_call, rho_put)

    return {
        "delta": _maybe_scalar(delta),
        "gamma": _maybe_scalar(gamma),
        "vega": _maybe_scalar(vega),
        "theta": _maybe_scalar(theta),
        "rho": _maybe_scalar(rho),
    }
```

Listing 4: Black-Scholes pricing and analytic Greeks.

```
def implied_volatility(
    market_price: Any,
    S: Any,
    K: Any,
    T: Any,
    r: Any,
    option_type: Any = "call",
    q: Any = 0.0,
    *,
    tol: float = 1e-8,
    max_iter: int = 120,
    max_vol: float = 10.0,
) -> float | np.ndarray:
    """Recover Black-Scholes implied volatility using robust bisection."""

    price, S, K, T, r, _, q = np.broadcast_arrays(
        np.asarray(market_price, dtype=float),
        np.asarray(S, dtype=float),
        np.asarray(K, dtype=float),
        np.asarray(T, dtype=float),
        np.asarray(r, dtype=float),
        np.asarray(0.0, dtype=float),
        np.asarray(q, dtype=float),
    )
    calls = np.broadcast_to(_is_call(option_type), price.shape)
    intrinsic = np.where(calls, np.maximum(S - K, 0.0), np.maximum(K - S, 0.0))
    valid = (price >= intrinsic - 1e-10) & (S > 0.0) & (K > 0.0) & (T > 0.0)
    out = np.full(price.shape, np.nan)
    if not np.any(valid):
```

```
        return _maybe_scalar(out)

    low = np.full(price.shape, 1e-8)
    high = np.full(price.shape, 1.0)
    for _ in range(12):
        model_high = np.asarray(bs_price(S, K, T, r, high, option_type, q))
        needs_higher = (model_high < price) & valid & (high < max_vol)
        if not np.any(needs_higher):
            break
        high = np.where(needs_higher, np.minimum(high * 2.0, max_vol), high)

    for _ in range(max_iter):
        mid = 0.5 * (low + high)
        model = np.asarray(bs_price(S, K, T, r, mid, option_type, q))
        low = np.where((model < price) & valid, mid, low)
        high = np.where((model >= price) & valid, mid, high)
        if np.nanmax(high[valid] - low[valid]) < tol:
            break

    out[valid] = 0.5 * (low[valid] + high[valid])
    return _maybe_scalar(out)
```

Listing 5: Black-Scholes implied-volatility inversion by robust bisection.

```
def crr_price(
    S: float,
    K: float,
    T: float,
    r: float,
    sigma: float,
    option_type: str = "call",
    *,
    steps: int = 250,
    style: OptionStyle = "european",
    q: float = 0.0,
) -> float:
    """Price a vanilla option with a CRR binomial tree."""

    if T <= 0.0:
        return max(S - K, 0.0) if option_type.lower().startswith("c") else max(K - S, 0.0)
    if steps < 1:
        raise ValueError("steps must be positive")
    if S <= 0.0 or K <= 0.0 or sigma <= 0.0:
        raise ValueError("S, K, and sigma must be positive")

    dt = T / steps
    u = math.exp(sigma * math.sqrt(dt))
    d = 1.0 / u
    growth = math.exp((r - q) * dt)
    p = (growth - d) / (u - d)
    if p < 0.0 or p > 1.0:
        raise ValueError("risk-neutral probability is outside [0, 1]")

    j = np.arange(steps + 1)
    terminal_spots = S * (u ** j) * (d ** (steps - j))
    is_call = option_type.lower().startswith("c")
    values = np.maximum(terminal_spots - K, 0.0) if is_call else np.maximum(K - terminal_spots, 0.0)
    discount = math.exp(-r * dt)

    for level in range(steps - 1, -1, -1):
        values = discount * (p * values[1 : level + 2] + (1.0 - p) * values[: level + 1])
        if style == "american":
            spots = S * (u ** np.arange(level + 1)) * (d ** (level - np.arange(level + 1)))
            exercise = np.maximum(spots - K, 0.0) if is_call else np.maximum(K - spots, 0.0)
            values = np.maximum(values, exercise)
    return float(values[0])


def convergence_table(
    S: float,
    K: float,
    T: float,
    r: float,
    sigma: float,
    option_type: str = "call",
    *,
```

```
    steps_grid: tuple[int, ...] = (25, 50, 100, 250, 500, 1000),
    q: float = 0.0,
) -> pd.DataFrame:
    """Return CRR convergence diagnostics against Black-Scholes."""

    benchmark = float(bs_price(S, K, T, r, sigma, option_type, q))
    rows = []
    for steps in steps_grid:
        price = crr_price(S, K, T, r, sigma, option_type, steps=steps, q=q)
        rows.append(
            {
                "steps": steps,
                "crr_price": price,
                "black_scholes_price": benchmark,
                "absolute_error": abs(price - benchmark),
                "relative_error": abs(price - benchmark) / max(benchmark, 1e-12),
            }
        )
    return pd.DataFrame(rows)
```

Listing 6: CRR binomial tree engine with European/American exercise support and convergence diagnostics.

```
@dataclass(frozen=True)
class MertonParams:
    """Risk-neutral Merton jump-diffusion parameters."""

    sigma: float
    jump_intensity: float
    jump_mean: float
    jump_vol: float

    @property
    def kappa(self) -> float:
        return math.exp(self.jump_mean + 0.5 * self.jump_vol * self.jump_vol) - 1.0


def _maybe_scalar(values: np.ndarray) -> float | np.ndarray:
    return float(values) if values.ndim == 0 else values


def _poisson_weights(lam_t: np.ndarray, max_jumps: int | None, tol: float) -> list[np.ndarray]:
    if max_jumps is None:
        lam_max = float(np.nanmax(lam_t)) if lam_t.size else 0.0
        max_jumps = max(20, int(math.ceil(lam_max + 8.0 * math.sqrt(lam_max + 1.0))))
    weights: list[np.ndarray] = []
    weight = np.exp(-lam_t)
    cumulative = weight.copy()
    weights.append(weight)
    for n in range(1, max_jumps + 1):
        weight = weight * lam_t / n
        cumulative = cumulative + weight
        weights.append(weight)
        if np.nanmax(1.0 - cumulative) < tol:
            break
    return weights


def merton_price(
    S: Any,
    K: Any,
    T: Any,
    r: Any,
    option_type: Any = "call",
    *,
    sigma: float,
    jump_intensity: float,
    jump_mean: float,
    jump_vol: float,
    max_jumps: int | None = None,
    poisson_tol: float = 1e-10,
) -> float | np.ndarray:
    """Price a European vanilla option under Merton jump diffusion.

    The implementation conditions on the number of jumps. Conditional on ``n``
```

```
    jumps, log-price remains Gaussian with adjusted drift and variance, which
    can be priced as a generalized Black-Scholes contract with continuous carry.
    """

    S, K, T, r, sigma, jump_intensity, jump_mean, jump_vol = np.broadcast_arrays(
        np.asarray(S, dtype=float),
        np.asarray(K, dtype=float),
        np.asarray(T, dtype=float),
        np.asarray(r, dtype=float),
        np.asarray(sigma, dtype=float),
        np.asarray(jump_intensity, dtype=float),
        np.asarray(jump_mean, dtype=float),
        np.asarray(jump_vol, dtype=float),
    )
    if np.all(jump_intensity <= 1e-14):
        return bs_price(S, K, T, r, sigma, option_type)
    if np.any(sigma <= 0.0) or np.any(jump_intensity < 0.0) or np.any(jump_vol < 0.0):
        raise ValueError("sigma and jump_vol must be positive; jump_intensity must be non-negative")

    lam_t = jump_intensity * np.maximum(T, 0.0)
    kappa = np.exp(jump_mean + 0.5 * jump_vol * jump_vol) - 1.0
    weights = _poisson_weights(lam_t, max_jumps, poisson_tol)
    price = np.zeros_like(S, dtype=float)

    for n, weight in enumerate(weights):
        total_var = sigma * sigma + (n * jump_vol * jump_vol) / np.maximum(T, 1e-16)
        sigma_n = np.sqrt(np.maximum(total_var, 1e-16))
        drift_n = r - jump_intensity * kappa + (n * jump_mean + 0.5 * n * jump_vol * jump_vol) / np.maximum(T,
      1e-16)
        q_n = r - drift_n
        price = price + weight * np.asarray(bs_price(S, K, T, r, sigma_n, option_type, q=q_n))
    return _maybe_scalar(price)
```

Listing 7: Merton jump-diffusion parameters and Poisson-mixture pricing engine.

```
def merton_greeks(
    S: float,
    K: float,
    T: float,
    r: float,
    option_type: str = "call",
    *,
    sigma: float,
    jump_intensity: float,
    jump_mean: float,
    jump_vol: float,
) -> dict[str, float]:
    """Finite-difference Greeks for the Merton model."""

    if jump_intensity <= 1e-14:
        return {key: float(value) for key, value in bs_greeks(S, K, T, r, sigma, option_type).items()}

    def price(s=S, k=K, t=T, rate=r, vol=sigma):
        return float(
            merton_price(
                s,
                k,
                max(t, 1e-8),
                rate,
                option_type,
                sigma=max(vol, 1e-8),
                jump_intensity=jump_intensity,
                jump_mean=jump_mean,
                jump_vol=jump_vol,
            )
        )

    h_s = max(1e-3, 1e-4 * S)
    h_v = max(1e-5, 1e-4 * sigma)
    h_r = 1e-4
    h_t = min(max(1e-5, 1e-4 * T), max(0.5 * T, 1e-5))

    base = price()
    up_s = price(s=S + h_s)
    dn_s = price(s=max(S - h_s, 1e-8))
    delta = (up_s - dn_s) / (2.0 * h_s)
```

```
    gamma = (up_s - 2.0 * base + dn_s) / (h_s * h_s)
    vega = (price(vol=sigma + h_v) - price(vol=max(sigma - h_v, 1e-8))) / (2.0 * h_v)
    rho = (price(rate=r + h_r) - price(rate=r - h_r)) / (2.0 * h_r)
    theta = (price(t=max(T - h_t, 1e-8)) - price(t=T + h_t)) / (2.0 * h_t)
    return {"delta": delta, "gamma": gamma, "vega": vega, "theta": theta, "rho": rho}


def simulate_merton_paths(
    S0: float,
    T: float,
    *,
    r: float,
    sigma: float,
    jump_intensity: float,
    jump_mean: float,
    jump_vol: float,
    steps: int = 365,
    paths: int = 1000,
    seed: int | None = 4331,
) -> np.ndarray:
    """Simulate Merton jump-diffusion paths under the risk-neutral measure."""

    rng = np.random.default_rng(seed)
    dt = T / steps
    kappa = math.exp(jump_mean + 0.5 * jump_vol * jump_vol) - 1.0
    out = np.empty((paths, steps + 1), dtype=float)
    out[:, 0] = S0
    drift = (r - jump_intensity * kappa - 0.5 * sigma * sigma) * dt
    diffusion_scale = sigma * math.sqrt(dt)
    for i in range(1, steps + 1):
        z = rng.standard_normal(paths)
        n_jumps = rng.poisson(jump_intensity * dt, paths)
        jump_log = rng.normal(jump_mean * n_jumps, jump_vol * np.sqrt(n_jumps))
        out[:, i] = out[:, i - 1] * np.exp(drift + diffusion_scale * z + jump_log)
    return out
```

Listing 8: Merton finite-difference Greeks and risk-neutral path simulation.

```
def estimate_historical_merton_params(
    index_history: pd.DataFrame,
    *,
    z_threshold: float = 3.0,
) -> tuple[MertonParams, pd.DataFrame]:
    """Estimate initial Merton parameters from historical index returns."""

    returns = detect_jumps(compute_log_returns(index_history), z_threshold=z_threshold)
    ppy = _periods_per_year(returns["timestamp"])
    years = max(len(returns) / ppy, 1e-12)
    non_jump_returns = returns.loc[~returns["is_jump"], "log_return"]
    jump_returns = returns.loc[returns["is_jump"], "log_return"]

    if len(non_jump_returns) < 3:
        non_jump_returns = returns["log_return"]
    sigma = float(non_jump_returns.std(ddof=1) * np.sqrt(ppy))
    sigma = float(np.clip(sigma, 0.03, 3.0))

    if len(jump_returns) == 0:
        jump_intensity = 0.25
        jump_mean = 0.0
        jump_vol = max(float(returns["log_return"].std(ddof=1) * 2.0), 0.01)
    else:
        jump_intensity = float(len(jump_returns) / years)
        jump_mean = float(jump_returns.mean())
        jump_vol = float(jump_returns.std(ddof=1)) if len(jump_returns) > 1 else abs(jump_mean)
        jump_vol = max(jump_vol, 0.01)

    params = MertonParams(
        sigma=sigma,
        jump_intensity=float(np.clip(jump_intensity, 1e-4, 250.0)),
        jump_mean=float(np.clip(jump_mean, -1.0, 1.0)),
        jump_vol=float(np.clip(jump_vol, 1e-4, 2.0)),
    )
    returns.attrs["periods_per_year"] = ppy
    return params, returns
```

Listing 9: Historical initialization of Merton parameters from index returns.

```python
def calibrate_merton_to_options(
    options: pd.DataFrame,
    initial: MertonParams,
    *,
    max_options: int = 250,
    rate_override: float | None = None,
) -> CalibrationResult:
    """Calibrate Merton parameters by minimizing weighted option price errors."""

    if minimize is None:  # pragma: no cover
        raise RuntimeError("scipy is required for calibration. Install requirements.txt first.")

    sample = calibration_sample(options, max_options=max_options)
    if sample.empty:
        return CalibrationResult(initial, np.nan, np.nan, 0, False, "No usable options")

    S = sample["underlying_price"].to_numpy(float)
    K = sample["strike"].to_numpy(float)
    T = sample["time_to_maturity"].to_numpy(float)
    market = sample["market_price_usd"].to_numpy(float)
    r = np.full(len(sample), rate_override, dtype=float) if rate_override is not None else sample["rate"].
     fillna(0.0).to_numpy(float)
    option_type = sample["option_type"].to_numpy(str)
    spread = sample.get("spread_usd", pd.Series(np.nan, index=sample.index)).fillna(0.0).to_numpy(float)
    weights = 1.0 / np.maximum(spread + 0.02 * market, 1.0)
    weights = weights / np.nanmean(weights)

    bounds = [(0.03, 3.0), (1e-4, 250.0), (-1.0, 1.0), (1e-4, 2.0)]
    x0 = np.array(
        [
            np.clip(initial.sigma, *bounds[0]),
            np.clip(initial.jump_intensity, *bounds[1]),
            np.clip(initial.jump_mean, *bounds[2]),
            np.clip(initial.jump_vol, *bounds[3]),
        ],
        dtype=float,
    )

    def objective(x: np.ndarray) -> float:
        sigma, jump_intensity, jump_mean, jump_vol = x
        model = np.asarray(
            merton_price(
                S,
                K,
                T,
                r,
                option_type,
                sigma=float(sigma),
                jump_intensity=float(jump_intensity),
                jump_mean=float(jump_mean),
                jump_vol=float(jump_vol),
            )
        )
        errors = (model - market) / np.maximum(market, 1.0)
        return float(np.nanmean(weights * errors * errors))

    result = minimize(objective, x0, method="L-BFGS-B", bounds=bounds, options={"maxiter": 300})
    params = MertonParams(
        sigma=float(result.x[0]),
        jump_intensity=float(result.x[1]),
        jump_mean=float(result.x[2]),
        jump_vol=float(result.x[3]),
    )
    model = np.asarray(
        merton_price(
            S,
            K,
            T,
            r,
            option_type,
            sigma=params.sigma,
            jump_intensity=params.jump_intensity,
            jump_mean=params.jump_mean,
            jump_vol=params.jump_vol,
        )
```

```
    )
    errors = model - market
    return CalibrationResult(
        params=params,
        rmse=float(np.sqrt(np.nanmean(errors * errors))),
        mae=float(np.nanmean(np.abs(errors))),
        n_options=len(sample),
        success=bool(result.success),
        message=str(result.message),
    )
```

Listing 10: Market calibration of Merton parameters to Deribit option prices.

```
def _surface_design(options: pd.DataFrame) -> tuple[np.ndarray, tuple[str, ...]]:
    x = options.get("log_moneyness")
    if x is None:
        x = np.log(options["strike"].to_numpy(float) / options["underlying_price"].to_numpy(float))
    else:
        x = x.to_numpy(float)
    t = np.maximum(options["time_to_maturity"].to_numpy(float), 1e-8)
    put = options["option_type"].astype(str).str.lower().str.startswith("p").to_numpy(float)
    short = (t <= 30.0 / 365.25).astype(float)
    sqrt_t = np.sqrt(t)
    # Three extra terms added for MR-ISVM:
    #   log_moneyness_x_sqrt_T : SVI-style cross term - controls how skew
    #                            steepness varies with maturity (key for term
    #                            structure of the crypto smile).
    #   put_x_sqrt_T           : puts and calls have different term structures;
    #                            this lets the surface tilt the IV term slope
    #                            separately for each side.
    #   log_moneyness_cube     : cubic asymmetry - crypto OTM puts are more
    #                            expensive than the quadratic term alone can
    #                            capture, driving systematic under-pricing.
    features = (
        "intercept",
        "log_moneyness",
        "abs_log_moneyness",
        "log_moneyness_sq",
        "sqrt_T",
        "T",
        "put",
        "put_x_log_moneyness",
        "short_maturity",
        "log_moneyness_x_sqrt_T",
        "put_x_sqrt_T",
        "log_moneyness_cube",
    )
    design = np.column_stack(
        [
            np.ones(len(options)),
            x,
            np.abs(x),
            x * x,
            sqrt_t,
            t,
            put,
            put * x,
            short,
            x * sqrt_t,          # SVI cross term
            put * sqrt_t,         # put-side term slope
            x * x * x,           # cubic skew asymmetry
        ]
    )
    return design, features


def fit_implied_surface(
    options: pd.DataFrame,
    *,
    max_options: int = 80,
    ridge: float = 1e-3,
) -> ImpliedSurfaceParams:
    """
    Fit a reduced MR-ISVM implied-volatility surface on a calibration sample.

    Changes from the original
```

```
    ------------------------
    Fix 1 - price-space objective from the start, no IV warm-start
        The original code fitted an OLS solution in IV space and used it as a
        warm-start for a price-space refinement.  The OLS warm-start lands in
        a local minimum of the IV loss, which is a different landscape from the
        price loss.  L-BFGS-B then does not move far enough to escape it.

        The new code initialises from a physically meaningful flat smile
        (intercept = median ATM IV, all other coefficients = 0) and runs
        L-BFGS-B directly on the price-space weighted RMSE objective with
        2 000 iterations.  This avoids the IV-space local minimum entirely.

    Fix 2 - vega-weighted objective instead of inverse-price weights
        The original weights = 1 / max(market_price, 0.5) give equal relative
        attention across strikes.  But USD RMSE is dominated by high-vega
        near-ATM options.  Fitting with inverse-price weights over-emphasises
        cheap deep-OTM options whose price errors are tiny in USD terms,
        causing the surface to under-fit ATM where the error budget matters.

        The new code weights each option by its approximate Black-Scholes vega
        (dC/dsigma approx S * phi(d1) * sqrtT).  This makes the calibration objective
        proportional to how much a given IV error contributes to USD price
        error, aligning the calibration loss with the evaluation metric.
    """
    sample = calibration_sample(options, max_options=max_options)
    sample = sample.dropna(subset=["mark_iv_decimal"]).copy()
    sample = sample[sample["mark_iv_decimal"].between(0.01, 4.0)]
    if sample.empty:
        raise ValueError("No usable options for implied-surface calibration")

    X, feature_names = _surface_design(sample)
    X = np.nan_to_num(X, nan=0.0, posinf=0.0, neginf=0.0)

    S_arr   = sample["underlying_price"].to_numpy(float)
    K_arr   = sample["strike"].to_numpy(float)
    T_arr   = np.maximum(sample["time_to_maturity"].to_numpy(float), 1e-8)
    r_arr   = sample["rate"].fillna(0.0).to_numpy(float)
    opt_arr = sample["option_type"].to_numpy(object)
    T_days  = T_arr * 365.25

    market_price = sample["market_price_usd"].to_numpy(float)

    sigma_floor = 0.20
    sigma_cap   = 3.0

    # ------------------------------------------------------------------
    # Fix 2: vega-based weights
    # ------------------------------------------------------------------
    # Approximate ATM sigma per option from mark_iv_decimal; fall back to
    # the sample median when the column is missing or NaN.
    sigma_atm = sample["mark_iv_decimal"].fillna(
        sample["mark_iv_decimal"].median()
    ).to_numpy(float)
    sigma_atm = np.clip(sigma_atm, 0.05, 3.0)

    # Black-Scholes vega approx S * phi(d1) * sqrtT  (independent of call/put)
    d1 = (
        np.log(S_arr / K_arr)
        + (r_arr + 0.5 * sigma_atm ** 2) * T_arr
    ) / (sigma_atm * np.sqrt(T_arr))
    vega_approx = S_arr * np.exp(-0.5 * d1 ** 2) / np.sqrt(2.0 * np.pi) * np.sqrt(T_arr)
    vega_approx = np.maximum(vega_approx, 1e-4)

    # Normalise so the average weight = 1 (keeps objective scale stable)
    weights = vega_approx / vega_approx.mean()
    weights = np.clip(weights, 0.05, 20.0)

    # ------------------------------------------------------------------
    # Helper: map coefficient vector -> clipped IV -> BS price array
    # ------------------------------------------------------------------
    def _price_from_coefs(c: np.ndarray) -> np.ndarray:
        sigma_iv = np.einsum("ij,j->i", X, c)
        adaptive_floor = np.maximum(
            sigma_floor,
            0.15 * np.sqrt(np.maximum(30.0 / T_days, 1.0)),
        )
```

```
        sigma_iv = np.clip(sigma_iv, adaptive_floor, sigma_cap)
        return np.asarray(bs_price(S_arr, K_arr, T_arr, r_arr, sigma_iv, opt_arr), dtype=float)

    # ------------------------------------------------------------------
    # Price-space objective (vega-weighted relative RMSE)
    # ------------------------------------------------------------------
    def objective(c: np.ndarray) -> float:
        model_px = _price_from_coefs(c)
        rel_err  = (model_px - market_price) / np.maximum(market_price, 0.5)
        return float(np.sqrt(np.average(rel_err ** 2, weights=weights)))

    # ------------------------------------------------------------------
    # Fix 1: physically meaningful warm-start - flat smile at median IV
    # ------------------------------------------------------------------
    # intercept = median mark IV of the calibration sample
    # all other coefficients = 0
    # This is a valid point in the price-space landscape (the surface
    # predicts a constant IV across all strikes and maturities equal to
    # the sample median) and avoids the IV-space local minimum that the
    # OLS warm-start introduces.
    atm_iv = float(np.median(sigma_atm))
    coefs_init = np.zeros(X.shape[1])
    coefs_init[0] = atm_iv          # intercept only

    if minimize is None:
        # scipy unavailable - return flat-smile solution
        model_px_flat = _price_from_coefs(coefs_init)
        price_rmse = float(np.sqrt(np.mean(
            ((model_px_flat - market_price) / np.maximum(market_price, 0.5)) ** 2
        )))
        return ImpliedSurfaceParams(
            tuple(float(v) for v in coefs_init),
            feature_names,
            sigma_floor=sigma_floor,
            fit_rmse=price_rmse,
        )

    # ------------------------------------------------------------------
    # Fix 1 continued: direct price-space optimisation, more iterations
    # ------------------------------------------------------------------
    # Ridge penalty applied inside the objective via a regularisation term
    # so the optimiser sees the penalised landscape from the first step.
    ridge_vec = np.full(X.shape[1], ridge)
    ridge_vec[0] = 0.0              # do not penalise the intercept

    def objective_regularised(c: np.ndarray) -> float:
        price_loss = objective(c)
        reg        = float(np.dot(ridge_vec, c ** 2))
        return price_loss + reg

    result = minimize(
        objective_regularised,
        coefs_init,
        method="L-BFGS-B",
        options={
            "maxiter": 2000,    # was 400 - needs more iterations from cold start
            "ftol":    1e-12,   # was 1e-10
            "gtol":    1e-7,    # gradient tolerance
        },
    )
    coefs_opt = result.x

    # Final price RMSE (unpenalised, matches summarize_candidate_errors metric)
    model_px_final = _price_from_coefs(coefs_opt)
    price_rmse = float(np.sqrt(np.mean(
        ((model_px_final - market_price) / np.maximum(market_price, 0.5)) ** 2
    )))

    return ImpliedSurfaceParams(
        tuple(float(v) for v in coefs_opt),
        feature_names,
        sigma_floor=sigma_floor,
        fit_rmse=price_rmse,
    )


```

```python
def predict_implied_surface_iv(options: pd.DataFrame, params: ImpliedSurfaceParams) -> np.ndarray:
    """Predict current-regime implied volatility for each option row.

    The adaptive floor replaces the fixed params.sigma_floor for very short
    maturities (T < 7 days). Near expiry, the polynomial surface can produce
    implausibly low IV in sparse strike regions; a time-scaled minimum of
    max(sigma_floor, 0.15 * sqrt(30/T_days)) prevents those artefacts from
    inflating pricing error on short-dated options without affecting longer
    maturities where the surface is well-identified.
    """

    X, _ = _surface_design(options)
    X = np.nan_to_num(X, nan=0.0, posinf=0.0, neginf=0.0)
    sigma = np.einsum("ij,j->i", X, np.asarray(params.coefficients, dtype=float))
    # Adaptive floor: tighter for short-dated options where the surface
    # extrapolates poorly, relaxed for longer maturities.
    T = np.maximum(options["time_to_maturity"].to_numpy(float), 1e-8)
    T_days = T * 365.25
    adaptive_floor = np.maximum(
        params.sigma_floor,
        0.15 * np.sqrt(np.maximum(30.0 / T_days, 1.0)),
    )
    return np.clip(sigma, adaptive_floor, params.sigma_cap)


def implied_surface_price(
    options: pd.DataFrame,
    params: ImpliedSurfaceParams,
    *,
    rate_override: float | None = None,
) -> np.ndarray:
    """Price options by plugging the fitted regime surface into Black-Scholes."""

    sigma = predict_implied_surface_iv(options, params)
    return np.asarray(
        bs_price(
            options["underlying_price"],
            options["strike"],
```

Listing 11: MR-ISVM 12-factor implied-volatility surface fit and Black-Scholes repricing.

```python
class RegimeLabel(enum.Enum):
    """Discrete market regimes used by the multi-regime MR-ISVM surface."""

    CALM   = "calm"    # low realised vol, no strong directional bias
    STRESS = "stress"  # high realised vol or crash-skew environment
    TREND  = "trend"   # intermediate vol with a persistent directional drift

    def __str__(self) -> str:
        return self.value


# Regime-classification thresholds (tuned to BTC/ETH empirical distributions).
_HIGH_VOL_THRESH    : float = 0.65   # annualised realised vol above -> STRESS
_CRASH_SKEW_THRESH  : float = -0.7   # return skewness below         -> STRESS
_TREND_DRIFT_THRESH : float = 0.40   # |annualised drift| above      -> TREND
_MIN_OPTIONS_PER_REGIME: int = 25    # fewer usable options -> skip, use fallback


@dataclass(frozen=True)
class MultiRegimeSurfaceParams:
    """
    One ImpliedSurfaceParams per detected regime plus the active regime label.

    Attributes
    ----------
    regime_surfaces : dict[RegimeLabel, ImpliedSurfaceParams]
        Fitted surface for each regime that had enough options to calibrate.
        A missing regime falls back to the CALM surface.
    current_regime  : RegimeLabel
        The regime detected from the most-recent returns at calibration time.
    fallback_label  : RegimeLabel
        Which surface to use when the requested regime is absent (default CALM).
    """

    regime_surfaces: dict[RegimeLabel, ImpliedSurfaceParams]
```

```
    current_regime:  RegimeLabel
    fallback_label:  RegimeLabel = RegimeLabel.CALM

    @property
    def active_params(self) -> ImpliedSurfaceParams:
        """ImpliedSurfaceParams for the current regime (with CALM fallback)."""
        return self.regime_surfaces.get(
            self.current_regime,
            self.regime_surfaces[self.fallback_label],
        )

    def params_for(self, label: RegimeLabel) -> ImpliedSurfaceParams:
        """Return the surface for an explicit regime (with fallback)."""
        return self.regime_surfaces.get(label, self.active_params)

    @property
    def n_regimes(self) -> int:
        """Number of distinct regimes that were successfully calibrated."""
        return len(self.regime_surfaces)

    def __repr__(self) -> str:
        labels = ", ".join(r.value for r in self.regime_surfaces)
        return (
            f"MultiRegimeSurfaceParams("
            f"regimes=[{labels}], "
            f"current={self.current_regime.value}, "
            f"n_regimes={self.n_regimes})"
        )


def detect_market_regime(
    return_frame: pd.DataFrame,
    *,
    lookback: int = 20,
    high_vol_thresh: float    = _HIGH_VOL_THRESH,
    crash_skew_thresh: float  = _CRASH_SKEW_THRESH,
    trend_drift_thresh: float = _TREND_DRIFT_THRESH,
) -> RegimeLabel:
    """
    Classify the current market into one of three regimes from recent returns.

    Parameters
    ----------
    return_frame : pd.DataFrame
        Must contain a "log_return" column.  ``attrs["periods_per_year"]`` is
        used for annualisation (default 365.25 for crypto).
    lookback : int
        Number of most-recent observations used for classification (default 30).
    high_vol_thresh : float
        Annualised realised-vol threshold above which the regime is STRESS.
    crash_skew_thresh : float
        Return skewness threshold below which the regime is STRESS.
    trend_drift_thresh : float
        |Annualised drift| threshold above which the regime is TREND.

    Returns
    -------
    RegimeLabel
        Priority: STRESS > TREND > CALM.
    """
    returns = return_frame["log_return"].dropna()
    if len(returns) < max(lookback // 2, 5):
        return RegimeLabel.CALM   # not enough history - safest default

    ppy    = float(return_frame.attrs.get("periods_per_year", 365.25))
    recent = returns.iloc[-lookback:]

    realized_vol     = float(recent.std(ddof=1)) * np.sqrt(ppy)
    skewness         = float(recent.skew())
    annualized_drift = float(recent.mean()) * ppy

    # STRESS: high vol or crash-type negative skew
    if realized_vol > high_vol_thresh or skewness < crash_skew_thresh:
        return RegimeLabel.STRESS

    # TREND: strong directional bias
```

```
    if abs(annualized_drift) > trend_drift_thresh:
        return RegimeLabel.TREND

    return RegimeLabel.CALM


def _label_option_regimes(
    options: pd.DataFrame,
    return_frame: pd.DataFrame,
    *,
    regime_window: int = 30,
) -> pd.Series:
    """
    Assign a RegimeLabel to every option row from the returns that preceded it.

    If options have no parseable date information, every row receives the
    current (most-recent) regime, so the multi-regime fit degrades gracefully
    to a single-regime calibration.

    Returns
    -------
    pd.Series[RegimeLabel]  - same index as `options`.
    """
    valid_returns = return_frame.dropna(subset=["log_return"]).copy()
    ppy = float(return_frame.attrs.get("periods_per_year", 365.25))

    # Build a date -> regime mapping over the return time series.
    if "datetime" in valid_returns.columns:
        ret_dates = pd.to_datetime(valid_returns["datetime"], utc=True, errors="coerce")
    elif "timestamp" in valid_returns.columns:
        ret_dates = pd.to_datetime(valid_returns["timestamp"], unit="ms", utc=True, errors="coerce")
    elif isinstance(valid_returns.index, pd.DatetimeIndex):
        ret_dates = pd.Series(pd.to_datetime(valid_returns.index, utc=True), index=valid_returns.index)
    else:
        current = detect_market_regime(return_frame)
        return pd.Series(current, index=options.index)

    date_mask = pd.Series(ret_dates, index=valid_returns.index).notna()
    valid_returns = valid_returns.loc[date_mask]
    ret_dates = pd.Series(ret_dates, index=valid_returns.index)
    if valid_returns.empty:
        current = detect_market_regime(return_frame)
        return pd.Series(current, index=options.index)

    date_to_regime: dict[pd.Timestamp, RegimeLabel] = {}
    log_returns = valid_returns["log_return"].to_numpy(float)

    for i, dt in enumerate(ret_dates):
        if i < regime_window // 2:
            label = RegimeLabel.CALM
        else:
            window_ret = log_returns[max(0, i - regime_window): i]
            tmp = pd.DataFrame({"log_return": window_ret})
            tmp.attrs["periods_per_year"] = ppy
            label = detect_market_regime(tmp, lookback=len(tmp))
        date_to_regime[dt] = label

    if not date_to_regime:
        current = detect_market_regime(return_frame)
        return pd.Series(current, index=options.index)

    sorted_dates = sorted(date_to_regime.keys())

    current = detect_market_regime(return_frame)

    def _nearest(opt_date: pd.Timestamp) -> RegimeLabel:
        if pd.isna(opt_date):
            return current
        idx = np.searchsorted(sorted_dates, opt_date, side="right") - 1
        return date_to_regime[sorted_dates[max(idx, 0)]]

    # Try to extract an observation date from the options DataFrame.
    opt_dates: pd.Series | None = None
    for col in ("valuation_datetime", "datetime", "date", "timestamp", "valuation_timestamp", "
     expiration_datetime", "expiry_date"):
        if col in options.columns:
```

```
            try:
                if col.endswith("timestamp") and pd.api.types.is_numeric_dtype(options[col]):
                    opt_dates = pd.to_datetime(options[col], unit="ms", utc=True, errors="coerce")
                else:
                    opt_dates = pd.to_datetime(options[col], utc=True, errors="coerce")
                break
            except Exception:
                pass
    if opt_dates is None and isinstance(options.index, pd.DatetimeIndex):
        opt_dates = pd.Series(options.index, index=options.index)

    if opt_dates is None:
        return pd.Series(current, index=options.index)

    return opt_dates.map(_nearest)


def fit_implied_surface_multi_regime(
    options: pd.DataFrame,
    return_frame: pd.DataFrame,
    *,
    max_options_per_regime: int = 60,
    ridge: float = 1e-3,
    regime_window: int = 30,
) -> MultiRegimeSurfaceParams:
    """
    Fit one ImpliedSurfaceParams per detected market regime (multi-regime MR-ISVM).

    This is the regime-aware companion to ``fit_implied_surface``.  It:

    1. Detects the current regime via ``detect_market_regime``.
    2. Labels every option row with the regime that prevailed on its date via
       ``_label_option_regimes``.
    3. Calls the **existing** ``fit_implied_surface`` independently for each
       regime slice that has >= ``_MIN_OPTIONS_PER_REGIME`` usable rows.
    4. Guarantees the CALM regime always has a surface (falls back to fitting
       on all options if the CALM slice is too small).

    Parameters
    ----------
    options : pd.DataFrame
        Full option universe for one currency.
    return_frame : pd.DataFrame
        Historical log-returns for the same currency.
    max_options_per_regime : int
        Forwarded to ``fit_implied_surface`` for each regime slice.
    ridge : float
        Ridge penalty forwarded to ``fit_implied_surface``.
    regime_window : int
        Rolling lookback (in return observations) for per-date labelling.

    Returns
    -------
    MultiRegimeSurfaceParams
        Call ``.active_params`` to get the ImpliedSurfaceParams for the current
        regime - fully compatible with ``price_candidate_models``.
    """
    current_regime = detect_market_regime(return_frame)
    regime_labels  = _label_option_regimes(options, return_frame, regime_window=regime_window)

    regime_surfaces: dict[RegimeLabel, ImpliedSurfaceParams] = {}

    for regime in RegimeLabel:
        slice_opts = options[regime_labels == regime].copy()
        usable = slice_opts.dropna(subset=["mark_iv_decimal"])
        usable = usable[usable["mark_iv_decimal"].between(0.01, 4.0)]

        if len(usable) < _MIN_OPTIONS_PER_REGIME:
            continue   # not enough data; this regime will use the CALM fallback

        try:
            regime_surfaces[regime] = fit_implied_surface(
                slice_opts,
                max_options=max_options_per_regime,
                ridge=ridge,
            )
```

```
606         except Exception:
607             continue   # degenerate slice - skip silently
608 
609     # Always guarantee a CALM surface as the universal fallback.
610     if RegimeLabel.CALM not in regime_surfaces:
611         try:
612             regime_surfaces[RegimeLabel.CALM] = fit_implied_surface(
613                 options,
614                 max_options=max_options_per_regime,
615                 ridge=ridge,
616             )
617         except Exception as exc:
618             raise RuntimeError(
619                 "MR-ISVM multi-regime: could not fit even the fallback CALM surface. "
620                 "Ensure the options DataFrame contains usable mark_iv_decimal values."
621             ) from exc
622 
623     return MultiRegimeSurfaceParams(
624         regime_surfaces=regime_surfaces,
625         current_regime=current_regime,
626         fallback_label=RegimeLabel.CALM,
627     )
628 
629 
630 def predict_implied_surface_iv_multi_regime(
631     options: pd.DataFrame,
632     params: MultiRegimeSurfaceParams,
633     *,
634     regime_override: RegimeLabel | None = None,
635 ) -> np.ndarray:
636     """
637     Predict IV using the regime-specific surface.
638 
639     Delegates to the existing ``predict_implied_surface_iv`` after selecting
640     the correct ImpliedSurfaceParams for the active (or overridden) regime.
641 
642     Parameters
643     ----------
644     regime_override : RegimeLabel or None
645         Force prediction from a specific regime's surface (e.g. for stress
646         testing).  When None (default), the current regime is used.
647     """
648     surface = params.params_for(regime_override) if regime_override is not None else params.active_params
649     return predict_implied_surface_iv(options, surface)
650 
651 
652 def implied_surface_price_multi_regime(
653     options: pd.DataFrame,
654     params: MultiRegimeSurfaceParams,
655     *,
656     rate_override: float | None = None,
657     regime_override: RegimeLabel | None = None,
658 ) -> np.ndarray:
659     """
660     Price options using the regime-aware MR-ISVM surface.
661 
662     Drop-in multi-regime replacement for ``implied_surface_price``.  Selects
663     the active regime's ImpliedSurfaceParams then delegates to the existing
664     ``implied_surface_price``, preserving all IV-clamping and adaptive-floor
665     logic exactly.
666 
667     Parameters
668     ----------
669     rate_override : float or None
670         Overrides the "rate" column for all options when supplied.
671     regime_override : RegimeLabel or None
672         Force a specific regime surface (e.g. for scenario analysis).
673     """
674     surface = params.params_for(regime_override) if regime_override is not None else params.active_params
675     return implied_surface_price(options, surface, rate_override=rate_override)
676 
677 
678 def summarize_regime_surfaces(params: MultiRegimeSurfaceParams) -> pd.DataFrame:
679     """
680     Tidy DataFrame comparing the fitted surfaces across all calibrated regimes.
681 
```

```
    Columns: regime, n_coefficients, fit_rmse, sigma_floor, sigma_cap, active.
    Purely for inspection - no effect on pricing.
    """
    rows = []
    for label, surface in params.regime_surfaces.items():
        rows.append({
            "regime":         label.value,
            "n_coefficients": len(surface.coefficients),
            "fit_rmse":       round(surface.fit_rmse, 6),
```

Listing 12: Regime detection and multi-regime MR-ISVM surface wrapper.

```
def _variance_state_from_returns(
    return_frame: pd.DataFrame,
    merton_params: MertonParams,
) -> SVCJMomentParams:
    r = return_frame["log_return"].dropna()
    ppy = float(return_frame.attrs.get("periods_per_year", 365.25))
    dt_approx = 1.0 / ppy

    rolling_var = r.rolling(20, min_periods=5).var() * ppy
    rolling_var = rolling_var.dropna()

    theta = float(np.clip(merton_params.sigma ** 2, 1e-6, 9.0))
    v0 = float(np.clip(
        rolling_var.iloc[-1] if not rolling_var.empty else theta,
        0.05 * theta, 9.0,
    ))

    acf = float(rolling_var.autocorr(lag=1)) if len(rolling_var) > 10 else 0.75
    kappa_v = float(np.clip(-np.log(np.clip(acf, 0.05, 0.98)) * ppy, 0.25, 12.0))

    jump_returns = return_frame.loc[
        return_frame.get("is_jump", pd.Series(False, index=return_frame.index)).astype(bool),
        "log_return",
    ]
    if len(jump_returns) > 0:
        variance_jump_mean = float(np.maximum(
            jump_returns.pow(2).mean() * ppy - theta,
            0.05 * theta,
        ))
    else:
        variance_jump_mean = 0.05 * theta
    variance_jump_mean = float(np.clip(variance_jump_mean, 1e-8, 4.0 * theta))

    if len(rolling_var) > 10:
        dv = rolling_var.diff().dropna()
        mean_v = float(rolling_var.mean())
        denom = np.sqrt(max(mean_v, 1e-8) * dt_approx * 20.0)
        xi = float(np.clip(dv.std() / denom, 0.05, 3.0))
    else:
        xi = 0.5

    variance_change = rolling_var.diff().dropna()
    aligned_returns = r.reindex(variance_change.index)
    if (
        len(variance_change) > 5
        and aligned_returns.std(ddof=1) > 0
        and variance_change.std(ddof=1) > 0
    ):
        leverage_rho = float(np.corrcoef(aligned_returns, variance_change)[0, 1])
    else:
        leverage_rho = -0.5

    return SVCJMomentParams(
        v0=v0,
        theta=theta,
        kappa_v=kappa_v,
        xi=float(np.clip(xi, 0.05, 3.0)),
        leverage_rho=float(np.clip(leverage_rho, -0.95, 0.95)),
        variance_jump_mean=variance_jump_mean,
        variance_multiplier=1.0,
        jump_intensity=merton_params.jump_intensity,
        jump_mean=merton_params.jump_mean,
        jump_vol=merton_params.jump_vol,
    )
```

```
854
855 def svcj_average_variance(
856     params: SVCJMomentParams,
857     T: np.ndarray | float,
858 ) -> np.ndarray:
859     """
860     Compute the risk-neutral expected average variance E[1/T integral0T vt dt]
861     for an SVCJ process, using the closed-form moment from the affine
862     Heston + jump framework.
863
864     Formula (Gatheral 2006, Ch. 2 + jump compensator)
865     -------------------------------------------------
866     For the CIR variance process with jumps:
867
868         E[vt] = theta + (v0 - theta) e^{-kappat}  +  lambda mu_v / kappa (1 - e^{-kappat})
869
870     Integrating over [0, T] and dividing by T:
871
872         avg_var = theta
873                 + (v0 - theta) (1 - e^{-kappaT}) / (kappaT)
874                 + lambda mu_v / kappa [1 - (1 - e^{-kappaT})/(kappaT)]
875
876     The vol-of-vol (xi) and leverage (leverage_rho) do not enter the
877     *first* moment of the variance, but they do affect the variance of
878     realised variance (the vol-of-vol risk premium).  We add a small
879     convexity correction term  +1/2 xi^2 sigma / kappa  (derived from the second
880     moment) so that higher xi produces a higher effective average variance,
881     consistent with the SVCJ implied-vol surface being wider than Heston.
882
883     Finally, variance_multiplier scales the result (calibrated to prices).
884     """
885     horizon = np.maximum(np.asarray(T, dtype=float), 1e-8)
886     kappa   = max(params.kappa_v, 1e-8)
887     lam     = max(params.jump_intensity, 0.0)
888     mu_v    = max(params.variance_jump_mean, 0.0)
889     xi      = params.xi
890     v0      = params.v0
891     theta   = params.theta
892
893     # mean-reversion decay averaged over [0, T]
894     decay_avg = (1.0 - np.exp(-kappa * horizon)) / (kappa * horizon)
895
896     # CIR mean path (no jumps)
897     base_avg = theta + (v0 - theta) * decay_avg
898
899     # variance-jump contribution to the mean
900     # lambda mu_v / kappa * [1 - (1 - e^{-kappaT})/(kappaT)]
901     jump_avg = lam * mu_v / kappa * (1.0 - decay_avg)
902
903     # vol-of-vol convexity correction:  +1/2 xi^2 * avg_var / kappa
904     # This is the leading term of Var(realised vol) that shifts the
905     # risk-neutral expectation of realised variance upward.
906     base_for_correction = np.maximum(base_avg + jump_avg, 1e-10)
907     xi_correction = 0.5 * xi ** 2 * base_for_correction / kappa
908
909     avg_var = (base_avg + jump_avg + xi_correction) * params.variance_multiplier
910     return np.maximum(avg_var, 1e-10)
911
912
913 def svcj_moment_price(
914     options: pd.DataFrame,
915     params: SVCJMomentParams,
916     *,
917     rate_override: float | None = None,
918 ) -> np.ndarray:
919     """
920     Price options with a moment-matched SVCJ approximation.
921
922     The effective diffusive volatility is taken from
923     ``svcj_average_variance`` (which now properly accounts for xi, the
924     variance-jump mean, and a convexity correction).  Jump risk is then
925     layered on top via the Merton series, using the calibrated jump
926     parameters - exactly the same structure as the original, but now
927     with a better sigma_eff.
928     """
929     avg_var   = svcj_average_variance(params, options["time_to_maturity"].to_numpy(float))
```

```
    sigma_eff = np.sqrt(avg_var)
    return np.asarray(
        merton_price(
            options["underlying_price"],
            options["strike"],
            options["time_to_maturity"],
            _option_rates(options, rate_override),
            options["option_type"],
            sigma=sigma_eff,
            jump_intensity=params.jump_intensity,
            jump_mean=params.jump_mean,
            jump_vol=params.jump_vol,
        )
    )


def calibrate_svcj_moment_proxy(
    options: pd.DataFrame,
    return_frame: pd.DataFrame,
    merton_params: MertonParams,
    *,
    max_options: int = 80,
) -> SVCJMomentParams:
    """
    Calibrate the SVCJ moment proxy to option prices.

    The price proxy depends on the average variance path, so the option
    calibration only refines parameters that enter that path directly:
    ``variance_multiplier`` and ``xi``. ``leverage_rho`` is still estimated
    from historical return/variance co-movement, but it is not optimized here
    because this moment-matched proxy does not use it in the pricing equation.
    """
    base = _variance_state_from_returns(return_frame, merton_params)
    sample = calibration_sample(options, max_options=max_options)
    if sample.empty or minimize is None:
        return base

    market = sample["market_price_usd"].to_numpy(float)
    weights = 1.0 / np.maximum(market, 0.5)

    def objective(x: np.ndarray) -> float:
        vm, xi = float(x[0]), float(x[1])
        params = replace(
            base,
            variance_multiplier=vm,
            xi=xi,
        )
        model = svcj_moment_price(sample, params)
        errors = (model - market) / np.maximum(market, 0.5)
        return float(np.sqrt(np.average(errors ** 2, weights=weights)))

    x0 = np.array([base.variance_multiplier, base.xi], dtype=float)
    bounds = [(0.25, 4.0), (0.05, 3.0)]
    result = minimize(
        objective,
        x0,
        method="L-BFGS-B",
        bounds=bounds,
        options={"maxiter": 300, "ftol": 1e-8},
    )
    vm_opt, xi_opt = (float(v) for v in result.x)
    return replace(
        base,
        variance_multiplier=vm_opt,
        xi=float(np.clip(xi_opt, 0.05, 3.0)),
    )
```

Listing 13: SVCJ moment proxy, identifiable calibration, and Merton-layered pricing.

```
def _initial_dynamic_jump_params(return_frame: pd.DataFrame, merton_params: MertonParams) -> DynamicJumpParams:

    jumps = return_frame.get("is_jump", pd.Series(False, index=return_frame.index)).astype(bool)
    ppy = float(return_frame.attrs.get("periods_per_year", 365.25))
    unconditional = max(float(jumps.mean()), 1.0 / max(len(jumps), 1))
    recent_window = max(20, int(round(ppy / 12.0)))
    recent = max(float(jumps.tail(recent_window).mean()), unconditional)
```

```
    current_scale = np.clip(recent / unconditional, 0.75, 4.0)
    return DynamicJumpParams(
        sigma=merton_params.sigma,
        base_intensity=float(max(1e-4, 0.65 * merton_params.jump_intensity)),
        current_intensity=float(max(1e-4, merton_params.jump_intensity * current_scale)),
        mean_reversion=6.0,
        jump_mean=merton_params.jump_mean,
        jump_vol=merton_params.jump_vol,
    )


def dynamic_effective_intensity(params: DynamicJumpParams, T: np.ndarray | float) -> np.ndarray:
    """Average the mean-reverting intensity over the option horizon."""

    horizon = np.maximum(np.asarray(T, dtype=float), 1e-8)
    beta = max(params.mean_reversion, 1e-8)
    decay_avg = (1.0 - np.exp(-beta * horizon)) / (beta * horizon)
    return np.maximum(params.base_intensity + (params.current_intensity - params.base_intensity) * decay_avg,
     1e-8)


def dynamic_jump_price(
    options: pd.DataFrame,
    params: DynamicJumpParams,
    *,
    rate_override: float | None = None,
) -> np.ndarray:
    """Price options under a Merton mixture with maturity-dependent intensity."""

    lambda_eff = dynamic_effective_intensity(params, options["time_to_maturity"].to_numpy(float))
    return np.asarray(
        merton_price(
            options["underlying_price"],
            options["strike"],
            options["time_to_maturity"],
            _option_rates(options, rate_override),
            options["option_type"],
            sigma=params.sigma,
            jump_intensity=lambda_eff,
            jump_mean=params.jump_mean,
            jump_vol=params.jump_vol,
        )
    )


def calibrate_dynamic_jump(
    options: pd.DataFrame,
    return_frame: pd.DataFrame,
    merton_params: MertonParams,
    *,
    max_options: int = 80,
) -> DynamicJumpParams:
    """Calibrate dynamic jump intensity parameters on the same option sample."""

    base = _initial_dynamic_jump_params(return_frame, merton_params)
    sample = calibration_sample(options, max_options=max_options)
    if sample.empty or minimize is None:
        return base
    market = sample["market_price_usd"].to_numpy(float)

    def objective(x: np.ndarray) -> float:
        sigma, base_intensity, current_intensity, mean_reversion = x
        if current_intensity < base_intensity:
            return 1e6 + (base_intensity - current_intensity) ** 2
        params = DynamicJumpParams(
            sigma=float(sigma),
            base_intensity=float(base_intensity),
            current_intensity=float(current_intensity),
            mean_reversion=float(mean_reversion),
            jump_mean=base.jump_mean,
            jump_vol=base.jump_vol,
        )
        model = dynamic_jump_price(sample, params)
        errors = (model - market) / np.maximum(market, 1.0)
        return float(np.mean(errors * errors))

```

```
    x0 = np.array([base.sigma, base.base_intensity, base.current_intensity, base.mean_reversion], dtype=float)
    bounds = [(0.03, 3.0), (1e-4, 250.0), (1e-4, 400.0), (0.25, 30.0)]
    result = minimize(objective, x0, method="L-BFGS-B", bounds=bounds, options={"maxiter": 250})
    return DynamicJumpParams(
        sigma=float(result.x[0]),
        base_intensity=float(result.x[1]),
        current_intensity=float(result.x[2]),
        mean_reversion=float(result.x[3]),
        jump_mean=base.jump_mean,
        jump_vol=base.jump_vol,
    )


def price_candidate_models(
    options: pd.DataFrame,
    *,
    historical_sigma: float,
    merton_params: MertonParams,
    surface_params: ImpliedSurfaceParams,
    svcj_params: SVCJMomentParams,
    dynamic_params: DynamicJumpParams,
    garch_params: GarchParams,
) -> pd.DataFrame:
    """Return long-form model prices and errors for the full option universe."""

    model_prices = {
        "Black-Scholes": np.asarray(
            bs_price(
                options["underlying_price"],
                options["strike"],
                options["time_to_maturity"],
                options["rate"].fillna(0.0),
                historical_sigma,
                options["option_type"],
            )
        ),
        "Merton": np.asarray(
            merton_price(
                options["underlying_price"],
                options["strike"],
                options["time_to_maturity"],
                options["rate"].fillna(0.0),
                options["option_type"],
                sigma=merton_params.sigma,
                jump_intensity=merton_params.jump_intensity,
                jump_mean=merton_params.jump_mean,
                jump_vol=merton_params.jump_vol,
            )
        ),
        "SVCJ proxy": svcj_moment_price(options, svcj_params),
        "MR-ISVM surface": implied_surface_price(options, surface_params),
        "Dynamic jump": dynamic_jump_price(options, dynamic_params),
        "GARCH variance": garch_price(options, garch_params),
    }

    rows = []
    base_cols = ["currency", "instrument_name", "option_type", "strike", "time_to_maturity", "log_moneyness"]
    market = options["market_price_usd"].to_numpy(float)
    for model, prices in model_prices.items():
        frame = options[base_cols].copy()
        frame["model"] = model
        frame["market_price_usd"] = market
        frame["model_price_usd"] = prices
        frame["error_usd"] = frame["model_price_usd"] - frame["market_price_usd"]
        frame["abs_error_usd"] = frame["error_usd"].abs()
        rows.append(frame)
    return pd.concat(rows, ignore_index=True)


def summarize_candidate_errors(priced: pd.DataFrame) -> pd.DataFrame:
    """Compute comparable pricing diagnostics for all candidate models."""

    return (
        priced.groupby(["currency", "model"], as_index=False)
        .agg(
            options=("abs_error_usd", "size"),
```

```
            bias_usd=("error_usd", "mean"),
            mae_usd=("abs_error_usd", "mean"),
            median_ae_usd=("abs_error_usd", "median"),
            rmse_usd=("error_usd", lambda s: float(np.sqrt(np.mean(np.square(s))))),
            p90_ae_usd=("abs_error_usd", lambda s: float(s.quantile(0.90))),
        )
        .sort_values(["currency", "mae_usd"])
        .reset_index(drop=True)
    )
```

Listing 14: Dynamic jump-intensity pricing and equal-level candidate error summaries.

```
def delta_hedge_short_option(
    path: Iterable[float],
    *,
    K: float,
    T: float,
    r: float,
    sigma: float,
    option_type: str = "put",
    model: str = "black_scholes",
    merton_params: dict[str, float] | None = None,
) -> dict[str, float]:
    """Simulate discrete delta hedging for a short option.

    The hedge uses only the current spot at each rebalance, then realizes the
    next price move. The final P&L is hedge portfolio value minus option payoff.
    """

    spots = np.asarray(list(path), dtype=float)
    if spots.ndim != 1 or len(spots) < 2:
        raise ValueError("path must contain at least two spot observations")
    dt = T / (len(spots) - 1)
    premium, delta = _model_price_delta(spots[0], K, T, r, sigma, option_type, model, merton_params)
    cash = premium - delta * spots[0]
    total_turnover = abs(delta * spots[0])

    for i in range(1, len(spots) - 1):
        cash *= math.exp(r * dt)
        tau = max(T - i * dt, 1e-8)
        _, new_delta = _model_price_delta(spots[i], K, tau, r, sigma, option_type, model, merton_params)
        trade = (new_delta - delta) * spots[i]
        cash -= trade
        total_turnover += abs(trade)
        delta = new_delta

    cash *= math.exp(r * dt)
    final_spot = spots[-1]
    payoff = max(final_spot - K, 0.0) if option_type.lower().startswith("c") else max(K - final_spot, 0.0)
    hedge_value = delta * final_spot + cash
    pnl = hedge_value - payoff
    return {
        "initial_spot": float(spots[0]),
        "final_spot": float(final_spot),
        "premium": float(premium),
        "payoff": float(payoff),
        "hedge_value": float(hedge_value),
        "pnl": float(pnl),
        "absolute_pnl": float(abs(pnl)),
        "turnover": float(total_turnover),
        "model": model,
    }


def run_hedge_experiment(
    paths: np.ndarray,
    *,
    K: float,
    T: float,
    r: float,
    sigma: float,
    option_type: str = "put",
    merton_params: dict[str, float] | None = None,
) -> pd.DataFrame:
    """Compare Black-Scholes and Merton hedge errors across simulated paths."""

```

```
    rows = []
    for path_id, path in enumerate(paths):
        for model in ("black_scholes", "merton"):
            rows.append(
                {
                    "path_id": path_id,
                    **delta_hedge_short_option(
                        path,
                        K=K,
                        T=T,
                        r=r,
                        sigma=sigma,
                        option_type=option_type,
                        model=model,
                        merton_params=merton_params,
                    ),
                }
            )
    return pd.DataFrame(rows)
```

Listing 15: Discrete delta-hedging simulation for short options.

```
def simulate_jump_hedge_experiment(
    S0: float,
    *,
    K: float,
    T: float,
    r: float,
    sigma: float,
    jump_intensity: float,
    jump_mean: float,
    jump_vol: float,
    steps: int = 60,
    paths: int = 250,
    seed: int = 4331,
    option_type: str = "put",
) -> pd.DataFrame:
    """Simulate jump paths and compare hedge error distributions."""

    simulated = simulate_merton_paths(
        S0,
        T,
        r=r,
        sigma=sigma,
        jump_intensity=jump_intensity,
        jump_mean=jump_mean,
        jump_vol=jump_vol,
        steps=steps,
        paths=paths,
        seed=seed,
    )
    return run_hedge_experiment(
        simulated,
        K=K,
        T=T,
        r=r,
        sigma=sigma,
        option_type=option_type,
        merton_params={
            "jump_intensity": jump_intensity,
            "jump_mean": jump_mean,
            "jump_vol": jump_vol,
        },
    )
```

Listing 16: Jump-path hedge experiment comparing Black-Scholes and Merton hedges.

C AI Assistance Statement

AI assistance was used to structure the project, implement Python modules, draft explanatory text, generate plots and tables, and format this report. The quantitative formulas, course alignment,

market data, statistical tests, and validation checks are explicit in the codebase and report. The implementation was verified with unit tests and live Deribit data smoke tests.